

Documentación Técnica – ICPC 2025

Universidad Antonio Nariño - Universidad Jorge Tadeo Lozano + Copilot

18 de octubre de 2025

Índice

1. Plantillas de Código	7
1.1. I/O rápido en C++	7
1.2. Macros utiles para el lenguaje	7
2. Lectura de casos	9
3. Estructuras de datos integradas en c++	12
4. Union-Find (DSU - Disjoint Set Union)	18
4.1. Descripción general	18
4.2. Complejidad	18
4.3. Aplicaciones comunes	19
4.4. Implementación clásica con compresión de caminos y unión por tamaño .	19
4.5. Variantes y extensiones	20
4.6. Ejemplo mínimo de uso	20
5. Segment Tree	20
5.1. Descripción general	20
5.2. Complejidad	20
5.3. Implementación básica (suma de rangos)	20
5.4. Segment Tree con Lazy Propagation	21
5.5. Transformaciones y variantes	22
5.6. Segment Tree Persistente	23
5.7. Aplicaciones comunes	24
6. Fenwick Tree (Binary Indexed Tree - BIT)	24
6.1. Descripción general	24
6.2. Complejidad	24
6.3. BIT básico para suma (point update, prefix query)	24
6.4. BIT con compresión de coordenadas	25
6.5. Transformaciones estándar	26
6.5.1. Range query con point update	26

6.5.2.	Point query con range update	26
6.5.3.	Range update con range query	26
6.6.	BIT para otras operaciones	27
6.6.1.	BIT para XOR	27
6.6.2.	BIT para mínimo/máximo	28
6.7.	K-ésimo elemento (order statistics) con BIT de frecuencias	28
6.8.	Inversiones en un arreglo (uso clásico con compresión)	29
6.9.	BIT 2D	29
7.	Sparse Table (RMQ)	30
7.1.	Descripción general	30
7.2.	Complejidad	30
7.3.	Idea principal	30
7.4.	Implementación para RMQ (mínimo en rango)	30
7.5.	Variantes	31
7.6.	Aplicaciones en programación competitiva	31
7.7.	Ejemplo de uso	32
8.	Hashing de Strings	32
8.1.	Descripción general	32
8.2.	Hash polinómico (rolling hash)	32
8.3.	Implementación de rolling hash	33
8.4.	Double hashing	33
8.5.	Aplicaciones comunes	34
8.6.	Hash bidireccional (forward y backward)	34
8.7.	Consideraciones avanzadas	34
8.8.	Ejemplo de uso	35
9.	Trie y Aho-Corasick	35
9.1.	Descripción general	35
9.2.	Complejidad	35
9.3.	Implementación básica de Trie	35
9.4.	Aho-Corasick	36
9.5.	Aplicaciones comunes	38
9.6.	Variantes y extensiones	38
9.7.	Ejemplo de uso	38
10.	BFS y DFS	39
10.1.	Descripción general	39
10.2.	Complejidad	39
10.3.	Implementación de BFS	39
10.4.	Implementación de DFS	39
10.5.	Variantes de DFS	40
10.6.	Aplicaciones comunes de BFS	40

10.7. Aplicaciones comunes de DFS	40
10.8. Ejemplo de uso combinado	40
11. Algoritmos de Caminos Mínimos: Dijkstra, Bellman-Ford y Floyd-Warshall	41
11.1. Descripción general	41
11.2. Dijkstra	41
11.2.1. Complejidad	41
11.2.2. Implementación	41
11.2.3. Aplicaciones	42
11.3. Bellman-Ford	42
11.3.1. Complejidad	42
11.3.2. Implementación	42
11.3.3. Aplicaciones	43
11.4. Floyd-Warshall	43
11.4.1. Complejidad	43
11.4.2. Implementación	43
11.4.3. Aplicaciones	43
11.5. Comparación	44
12. Árbol de Expansión Mínima: Kruskal y Prim	44
12.1. Descripción general	44
12.2. Algoritmo de Kruskal	44
12.2.1. Idea principal	44
12.2.2. Complejidad	44
12.2.3. Implementación	44
12.2.4. Aplicaciones	45
12.3. Algoritmo de Prim	45
12.3.1. Idea principal	45
12.3.2. Complejidad	45
12.3.3. Implementación	46
12.3.4. Aplicaciones	46
12.4. Comparación entre Kruskal y Prim	46
13. Longest Increasing Subsequence (LIS)	47
13.1. Descripción general	47
13.2. Complejidad	47
13.3. Implementación en $\mathcal{O}(n^2)$	47
13.4. Implementación en $\mathcal{O}(n \log n)$	47
13.5. Reconstrucción de la subsecuencia	48
13.6. Aplicaciones comunes	48
13.7. Variantes avanzadas	48
13.8. Ejemplo de uso	49

14.DP con Bitmasks	49
14.1. Descripción general	49
14.2. Complejidad	49
14.3. Ejemplo clásico: Traveling Salesman Problem (TSP)	49
14.3.1. Definición de estado	49
14.3.2. Recurrencia	50
14.3.3. Implementación	50
14.4. Ejemplo: Subset Sum con bitmask DP	50
14.5. Técnicas avanzadas	51
14.6. Aplicaciones comunes	51
14.7. Ejemplo de uso: Assignment Problem	51
15.DP en Árboles	52
15.1. Descripción general	52
15.2. Patrones comunes de Tree DP	52
15.3. Ejemplo 1: Tamaño de subárboles	52
15.4. Ejemplo 2: DP de caminos más largos (diámetro del árbol)	53
15.5. Ejemplo 3: Independent Set en árboles	53
15.6. Ejemplo 4: Rerooting DP	54
15.7. Aplicaciones comunes	54
15.8. Ejemplo de uso	55
16.DP en Cadenas: Subsecuencias, Substrings y Palíndromos	55
16.1. Notas generales de compilación	55
16.2. Longest Common Subsequence (LCS)	55
16.3. Longest Common Substring	56
16.4. Longest Palindromic Subsequence (LPS)	56
16.5. Longest Palindromic Substring	57
16.6. Edit Distance (Levenshtein)	57
16.7. Notas de optimización y robustez	58
17.Geometría Computacional General	58
17.1. Descripción general	58
17.2. Representación de puntos y vectores	58
17.3. Operaciones básicas	59
17.4. Orientación de tres puntos	59
17.5. Área de un polígono (fórmula de Shoelace)	59
17.6. Distancia punto a recta	60
17.7. Proyección de punto sobre recta	60
17.8. Circunferencia	60
17.9. Triángulos	60
17.10 Intersección de segmentos	60
17.11 Convexidad	60
17.12 Métodos útiles	61

17.13	Precisión numérica	61
18.	Conteo Combinatorio con Programación Dinámica	61
18.1.	Descripción general	61
18.2.	Técnicas fundamentales	61
18.3.	Precomputación combinatoria	62
18.3.1.	Factoriales y combinaciones	62
18.4.	Ejemplo 1: Número de formas de sumar a n con enteros positivos	62
18.5.	Ejemplo 2: Número de arreglos válidos (permutaciones con restricciones)	63
18.6.	Ejemplo 3: Número de formas de colocar k objetos en n cajas sin vacías	63
18.7.	Ejemplo 4: Caminos en una cuadrícula	63
18.8.	Principios combinatorios útiles	64
19.	Backtracking y Perfección de la Fuerza Bruta	64
19.1.	Descripción general	64
19.2.	Esquema general	65
19.3.	Técnicas para perfeccionar fuerza bruta	65
19.4.	Ejemplo 1: N-Reinas	65
19.5.	Ejemplo 2: Generar todas las permutaciones	66
19.6.	Ejemplo 3: Subconjuntos con suma objetivo	66
19.7.	Aplicaciones comunes	66
19.8.	Consejos prácticos	67
20.	Aritmética Modular y Teoría de Números	67
20.1.	Aritmética modular	67
20.1.1.	Propiedades básicas	67
20.1.2.	Inverso modular	67
20.2.	Exponenciación rápida	68
20.3.	Pequeño teorema de Fermat	68
20.4.	Teorema de Euler	68
20.4.1.	Función totiente	68
20.5.	Criba de Eratóstenes	69
20.6.	Factorización en primos	69
20.7.	Conteo de divisores	69
20.8.	Suma de divisores	69
20.9.	Aplicaciones comunes	69
21.	Operaciones Bit a Bit y Aplicaciones	70
21.1.	Descripción general	70
21.2.	Operaciones básicas	70
21.3.	Operaciones útiles	70
21.4.	Aplicaciones comunes	71
21.5.	Ejemplo: recorrer todos los subconjuntos de una máscara	71
21.6.	Ejemplo: contar bits encendidos	71

21.7. Ejemplo: encontrar el bit más bajo encendido	71
21.8. Ejemplo: generar todas las combinaciones de k bits encendidos en n posi- ciones	71
21.9. Consejos prácticos	72
22.Fórmulas Generales y Teoremas Clásicos	72
22.1. Áreas y perímetros	72
22.2. Trigonometría esencial	73
22.3. Álgebra y combinatoria	73
22.4. Teoremas clásicos	73
22.5. Fórmulas de conteo	74
22.6. Fórmulas físicas discretas útiles	74
22.7. Constantes útiles	75
22.8. Aplicaciones comunes	75
22.9. Detalles, variantes y consideraciones avanzadas	75
23.Tablas de Complejidad	76
23.1. Estructuras estándar	76
23.2. Algoritmos comunes	77
24.Fórmulas Matemáticas	77
24.1. Sumatorias clásicas	77
24.2. Identidades algebraicas	77
24.3. Fórmulas geométricas	78
24.4. Combinatoria	78
24.5. Trigonometría	79
24.6. Teoría de números	79
25.Cómo abordar problemas algorítmicos	79
25.1. 1. Leer con atención	79
25.2. 2. Identificar la categoría del problema	79
25.3. 3. Analizar restricciones	80
25.4. 4. Pensar en estados y transiciones	80
25.5. 5. Escribir código limpio y modular	80
25.6. 6. Depurar con propósito	80
25.7. 7. Optimizar solo si es necesario	80
25.8. 8. Aprender del problema	81

1. Plantillas de Código

1.1. I/O rápido en C++

Se encargan de desincronizar el cin y el cout con el fin de eliminar el overhead y alcanzar una velocidad similar al printf y scanf de C.

```
1 ios::sync_with_stdio(false);
2 cin.tie(nullptr);
```

1.2. Macros utiles para el lenguaje

a. Atajos de sintaxis

```
1 #define pb push_back //metodo de insercion comun en vectores
2 #define mp make_pair //construccion de pares en ejecucion
   para maps
3 #define all(x) (x).begin(), (x).end() //util para sort y
   otros metodos que requieren iteradores inicio y fin
4 #define rall(x) (x).rbegin(), (x).rend() //igual que all
   pero en orden inverso
5 #define sz(x) (int)((x).size()) //longitud de las
   principales DS std
```

b. Tipos de datos frecuentes

```
1 #define ll long long //entero 64 bits (maximo espacio c++)
2 #define ull unsigned long long //igual a ll, para casos sin
   negativos
3 //vectores de los tipos principales
4 #define vi vector<int>
5 #define vll vector<long long>
6 #define vd vector<double>
7 #define vs vector<string>
```

c. Bucles rapidos

```
1 #define FOR(i,a,b) for(long long i=(a); i<(b); i++) //Bucle
   for 1 a 1 desde a hasta b
2 #define REP(i,n) for(long long i=0; i<(n); i++) //Bucle
   for 1 a 1 de 0 a n (Ideal para recorrer todo un vector)
3 #define ROF(i,a,b) for(long long i=(a); i>=(b); i--) //Bucle
   descendente 1 a 1 de a hasta b
```

d. Constantes

```
1 #define INF 1000000000 //Posible infinito para int
2 #define LINF 1000000000000000000LL //Posible infinito para
   long long
```

```

3  #define MOD    1000000007 //Constante 10^9 + 7 para problemas
    gigantes
4  #define MOD2  998244353 //Otra constante modular menos usada
5  #define EPS    1e-9 //Margen de error para operaciones punto
    flotante

```

e. Debugging rapido (No sabemos debuggear)

```

1  #define debug(x) cerr << #x << " = " << (x) << endl //Se usa
    cerr ya que no es evaluado por el juez
2  #define debugv(v) { cerr << #v << " = "; for(auto _ : v)
    cerr << _ << " "; cerr << endl; }

```

f. Conversiones rapidas (Tipos y bases)

```

1  #define toStr(x) to_string(x) //Conversion numero a string
2  //String a numeros de diferentes tamanos
3  #define toInt(x) stoi(x) //A int
4  #define toLL(x) stoll(x) //A long long
5  #define toD(x) stod(x) //A double
6  //Conversiones de notaciones
7  #define toBin(x, n) bitset<n>(x).to_string() //Entero a
    string binario de n bits (n debe ser constante)
8  #define toBin32(x) bitset<32>(x).to_string() //Entero a
    string binario de 32 bits (util para int)
9  #define binToDec(x) stoi(x, nullptr, 2) //String binario a
    entero(int)
10 #define hexToDec(x) stoi(x, nullptr, 16) //String
    hexadecimal a entero
11 #define toHex(x) ([&(int v){ stringstream ss; ss<<hex<<v;
    return ss.str(); })(x) //Decimal(int) a notacion
    hexadecimal como string

```

g. Operaciones de bits (Con long long - complejidad O(1))

```

1  #define checkBit(x,i) ((x) & (1LL<<(i))) //Verificar bit i
    encendido
2  #define setBit(x,i) ((x) | (1LL<<(i))) //Encender bit i
3  #define clearBit(x,i) ((x) & ~(1LL<<(i))) //Apagar bit i
4  #define flipBit(x,i) ((x) ^ (1LL<<(i))) //Cambiar estado
    del bit inmediatamente (0 a 1 o 1 a 0)
5  #define getBit(x,i) (((x)>>(i)) & 1LL) //Obtener valor del
    bit
6  #define popcount(x) __builtin_popcountll(x) //Contar
    cantidad de bits en 1
7  #define lowbit(x) ((x) & (-x)) //Obtener indice del
    bit menos significativo encendido (x > 0)
8  #define msbIndex(x) (63 - __builtin_clzll(x)) //Longitud
    de numero en bits (x > 0)

```


2. Lectura de casos

Para la lectura de la entrada del programa se presentan principalmente 3 tipos de entrada:

- a. Numero de casos al inicio: Su principal característica se encuentra en que la primera línea nos dice la cantidad de casos que vamos a recibir a continuación, independientemente de si estos casos van a ser números, letras o bloques estructurados de información

- Ejemplo de entrada:

```
3 -> Numero de casos
10 -> Caso 1
20 -> Caso 2
30 -> Caso 3
```

- Recepción ideal:

Lo mejor para estos casos es recibir en una variable la cantidad de casos y luego con un bucle iterar recibiendo en cada caso la información e imprimiendo en el StdOut la respuesta (Nunca acumular la respuesta de múltiples casos para arrojarla toda al final)

```
1      #include<bits/stdc++.h>
2      using namespace std;
3
4      int main() {
5          int numCases;
6          cin>>numCases;
7          for(int i = 0; i < numCases; i++){
8              //Se recibe cada caso, se procesa y se da la
               respuesta inmediatamente
9              int case_i;
10             cin>>case_i;
11             cout<<case_i<<endl;
12         }
13     }
```

- b. Casos hasta que se cumpla una condición: Se entregan casos en cada línea (O en bloques de líneas), hasta que se encuentre un carácter especial de finalización (0, -1, END, etc)

- Ejemplo de entrada:

```
2          -> Caso 1: Se recibirá un vector con 2 elementos
1 2          -> Vector del caso 1
2          -> Caso 2: Se recibirá otro vector con 2 elementos
3 4          -> Vector del caso 2
```

```

4      -> Caso 3: Se recibirá otro vector, esta vez de 4 elementos
1 2 3 4 -> Vector del caso 3
0      -> Fin de entrada dado por el 0

```

- Recepción ideal:

Para estos casos lo mas común es utilizar un bucle do while o un while que incluya la recepción dentro de su condición, de esta forma podemos procesar uno a uno los casos, ir imprimiendo su salida y garantizar que frene sin imprimir valores extraños al recibir el caracter de finalización

```

1      #include<bits/stdc++.h>
2      using namespace std;
3
4      int main() {
5          int tamVec;
6          while(
7              cin>>tamVec //Recepcion dentro del condicional
8              && tamVec != 0 //Validacion de las entradas
9                  recibidas inmediatamente
10             ){
11                 //Creacion del vector de tamaño tamVec con valor
12                 inicial 0
13                 vector<int> values(tamVec, 0);
14                 //Recepcion de los elementos del vector
15                 for(int i = 0; i < tamVec; i++){
16                     cin>>values[i];
17                 }
18                 //Impresion inmediata de la respuesta del caso
19                 cout<<"[ ";
20                 for(int value: values){
21                     cout<<value<<" ";
22                 }
23                 cout<<"]"<<endl;
24             }
25     }

```

- c. Casos hasta que se termine la entrada: No existe una indicación de final dentro de la entrada, el programa debe finalizar exactamente cuando ya no hayan mas casos de entrada

- Ejemplo de entrada:

```

10     -> Caso 1
20     -> Caso 2
30     -> Caso 3
40     -> Caso 4

```

- Recepción ideal:

Para estos casos lo mas optimo (Y casi necesario) consiste en meter la lecutra de entrada dentro de un bucle while, si no sabemos la cantidad de valores exacta que vamos a recibir en cada caso, habrá que utilizar obligatoriamente los flags de fin de linea de cin, o getline si estrictamente cada linea es un caso

```

1      #include <bits/stdc++.h>
2      using namespace std;
3
4      int main() {
5          int num;
6          while(cin>>num){ //Bucle se ejecuta hasta que ya no
                          pueda recibir mas datos
7              cout<<"Respuesta: "<<num<<endl; //Respuesta
                          inmediata del caso
8          }
9      }
```

- d. Casos con texto o cantidad variable de datos por línea: Cada línea de la entrada representa un caso independiente, y puede contener desde texto libre hasta una cantidad arbitraria de números. En este escenario lo más sencillo es leer la línea completa como cadena y luego procesarla según la lógica del problema.

- Ejemplo de entrada:

```

Hola mundo
1 2 3
Esto es un caso
10 20 30 40
```

- Recepción ideal:

Se recomienda usar `getline` para capturar la línea completa. Si se necesitan números, se puede pasar la línea a un `stringstream` y extraerlos; si es texto, se procesa directamente.

```

1      #include <bits/stdc++.h>
2      using namespace std;
3
4      int main() {
5          string line;
6          while (getline(cin, line)) { // Leer cada linea
                                      hasta EOF
7              if (line.empty()) continue; // Saltar lineas
                                      vacias
8
9              // Procesar la linea (ejemplo: imprimirla)
10             cout << "Caso: " << line << endl;
11         }
```

3. Estructuras de datos integradas en c++

Las estructuras de datos de la STL (*Standard Template Library*) son contenedores genéricos que permiten almacenar y manipular colecciones de elementos de manera eficiente. Su gran ventaja es que ya vienen implementadas y optimizadas, por lo que el programador solo debe preocuparse por usarlas correctamente.

Concepto de iteradores

Un **iterador** es un objeto que actúa como un puntero generalizado:

- Permite recorrer los elementos de un contenedor (vector, lista, set, etc.).
- Se obtiene normalmente con los métodos `begin()` y `end()` de la estructura.
- Se avanza con el operador `++` y se accede al valor con el operador `*`, de la misma forma que los punteros `int`.

Es importante conocer su concepto debido a que algunas estructuras de datos que no poseen elementos contiguos en memoria no son accesibles por índice (ej. `set`), por lo que es necesario usar los iteradores para recorrer el contenido. Ejemplo básico:

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     vector<int> v = {10, 20, 30};
6
7     // Recorrido con iterador
8     for (auto it = v.begin(); it != v.end(); it++) {
9         cout << *it << " "; // imprime 10 20 30
10    }
11 }
```

Ventajas de los iteradores

- Unifican la forma de recorrer diferentes estructuras (no importa si es un vector, una lista o un set).
- Permiten usar algoritmos genéricos de la STL (`sort`, `find`, `lower_bound`, etc.).

Tipos de iteradores

- `begin()` y `end()`: recorren de inicio a fin.
- `rbegin()` y `rend()`: recorren en orden inverso.
- Const-iteradores (`cbegin()`, `cend()`): solo lectura.

Ejemplo con recorrido inverso:

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     vector<int> v = {10, 20, 30};
6
7     for (auto it = v.rbegin(); it != v.rend(); ++it) {
8         cout << *it << " "; // imprime 30 20 10
9     }
10 }
```

Stack (pila)

- Estructura LIFO (Last In, First Out).
- Operaciones principales:
 - `push(x)`: insertar elemento.
 - `pop()`: eliminar el último insertado.
 - `top()`: acceder al último insertado.
 - `empty()`: verificar si está vacía.
- Importante: Esta estructura de datos no expone los iteradores, por lo que no se puede recorrer para conservar su patrón LIFO
- Ejemplo:

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     stack<int> st; //Se crea una pila que contendrá enteros
6     st.push(10); //Se inserta 10
7     st.push(20);
8     cout << st.top() << "\n"; //Se observa el valor del elemento
9                               // en la cima (20)
10    st.pop(); //Se saca el valor de la cima (No retorna el valor
11              // , debe ser consultado antes de sacarlo)
12    cout << st.top() << "\n"; // 10
13 }
```

Queue (cola)

- Estructura FIFO (First In, First Out).
- Operaciones principales:
 - `push(x)`: insertar al final.
 - `pop()`: eliminar el primero.
 - `front()`: acceder al primero.
 - `back()`: acceder al último.
- Importante: Esta estructura de datos no expone los iteradores, por lo que no se puede recorrer para conservar su patrón FIFO
- Ejemplo:

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     queue<int> q; //Se crea una cola de enteros
6     q.push(1); //Se insertan los elementos
7     q.push(2);
8     cout << q.front() << "\n"; // Se consulta el elemento del
        frente (1)
9     q.pop(); //Se saca el elemento del frente (No lo retorna)
10    cout << q.front() << "\n"; //2
11 }
```

Deque (doble cola)

- Permite inserción y borrado tanto al frente como al final.
- Operaciones principales:
 - `push_front(x)`, `push_back(x)`
 - `pop_front()`, `pop_back()`
 - `front()`, `back()`
- Nota: las estructuras `queue` y `stack` funcionan en base a este contenedor, por lo que si se requiere una pila o cola con posibilidad de recorrido, puede implementarse fácilmente con esta estructura
- Ejemplo:

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
```

```

4  int main() {
5      deque<int> d; //Se crea la deque para enteros
6      d.push_back(5); //Se inserta 5 al final
7      d.push_front(3); //Se inserta 3 al frente
8      cout << d.front() << " " << d.back() << "\n"; // 3 5
9  }

```

Priority Queue (cola de prioridad)

- Cola que siempre devuelve el mayor (por defecto) o menor (si se define comparador) elemento en el frente.
- Implementada como heap binario.
- Operaciones principales:
 - push(x): insertar.
 - pop(): eliminar el mayor/menor.
 - top(): acceder al mayor/menor.
- Ejemplo:

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      priority_queue<int> pq; // max-heap (Mayor elemento al
6      frente)
7      pq.push(10);
8      pq.push(5);
9      pq.push(20);
10     cout << pq.top() << "\n"; // 20 (Elemento mayor)
11     pq.pop(); //Se saca el mayor
12     cout << pq.top() << "\n"; // 10 (Nuevo elemento mayor)
13
14     // min-heap (Menor elemento al frente)
15     priority_queue<int, vector<int>, greater<int>> minpq; //Se
16     usa greater para que el elemento mas grande quede al
17     fondo
18     minpq.push(10);
19     minpq.push(5);
20     cout << minpq.top() << "\n"; // 5
21 }

```

Set y multiset

- **set**: Contenedor ordenado que almacena elementos **únicos**, en orden ascendente por defecto (usa un árbol balanceado tipo Red-Black Tree).

- **multiset**: Igual que **set**, pero permite elementos repetidos.
- Operaciones principales:
 - **insert(x)**: inserta un elemento.
 - **erase(x)**: elimina un elemento (o todos los iguales en multiset).
 - **find(x)**: busca un elemento. (Devuelve el iterador que apunta a el elemento o **end()** si no se encuentra)
 - **count(x)**: cuenta ocurrencias (0 o 1 en **set**, ≥ 0 en **multiset**).

■ Ejemplo:

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     set<int> s;
6     s.insert(5);
7     s.insert(2);
8     s.insert(5); // ignorado, ya existe
9     //Se puede usar el for de elementos, no existen indices
10    for (int x : s) {
11        cout << x << " "; // 2 5
12    }
13    cout << "\n";
14
15    multiset<int> ms;
16    ms.insert(5);
17    ms.insert(5);
18    cout << ms.count(5) << "\n"; // Cuenta 2 elementos
19 }
```

Unordered_set

- Conjunto no ordenado, implementado con **hashing** (Los tipos primitivos y string de c++ son hashables, cualquier otro tipo de dato requiere implementar hasing personalizado).
- Almacena elementos únicos, pero sin orden definido.
- Operaciones principales: **insert**, **erase**, **find**, **count**.
- Ventaja: Operaciones en $O(1)$
- Ejemplo:

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
```



```

4 int main() {
5     unordered_set<string> us;
6     us.insert("hola");
7     us.insert("mundo");
8     //Se busca un elemento
9     if (us.find("hola") != us.end()){
10         cout << "Encontrado\n";
11     }
12 }

```

Map y multimap

- map: Diccionario ordenado (clave → valor), implementado con árbol balanceado.
- multimap: Igual que map, pero permite claves repetidas.
- Las claves se mantienen ordenadas automáticamente.
- Operaciones principales:
 - m[key] = value: inserta o actualiza.
 - insert({key, value}): inserta par clave-valor.
 - find(key): busca clave.
 - erase(key): elimina por clave.
 - count(key) : Cuenta por clave (multimap)
- Ejemplo:

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     map<string,int> m;
6     m["uva"] = 10;
7     m["pera"] = 20;
8     for (auto [k,v] : m)
9         cout << k << " -> " << v << "\n";
10    // salida ordenada por clave: pera -> 20, uva -> 10
11
12    multimap<int,string> mm;
13    mm.insert({1,"A"});
14    mm.insert({1,"B"});
15    cout << mm.count(1) << "\n"; // 2
16 }

```

Unordered_map

- Diccionario no ordenado, implementado con **hashing**.
- Claves únicas, acceso promedio $O(1)$.
- Operaciones principales: `m[key]`, `insert`, `find`, `erase`.
- Ejemplo:

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     unordered_map<string,int> um;
6     um["uva"] = 10;
7     um["pera"] = 20;
8     //Se busca comparando con el iterador end() que indica
        elemento no encontrado
9     if (um.find("uva") != um.end()){
10         cout << "uva -> " << um["uva"] << "\n";
11         //Mismo resultado que con map, mas rapido.
12     }
13 }
```

4. Union-Find (DSU - Disjoint Set Union)

4.1. Descripción general

La estructura de datos Union-Find, también conocida como DSU (Disjoint Set Union), permite mantener un conjunto de elementos particionados en subconjuntos disjuntos. Soporta dos operaciones fundamentales:

- `find(x)`: Retorna el representante (líder) del conjunto al que pertenece el elemento `x`.
- `unite(x, y)`: Une los conjuntos que contienen a `x` y `y`.

Estas operaciones se pueden implementar de forma eficiente utilizando compresión de caminos y unión por tamaño o por rango.

4.2. Complejidad

- `find` y `unite`: $\mathcal{O}(\alpha(n))$ amortizado, donde α es la inversa de la función de Ackermann (crece extremadamente lento).

4.3. Aplicaciones comunes

- Detección de ciclos en grafos no dirigidos.
- Construcción de árboles de expansión mínima (Kruskal).
- Agrupación de componentes conexas.
- Procesamiento de consultas de conectividad dinámica.

4.4. Implementación clásica con compresión de caminos y unión por tamaño

```
1 struct DSU {
2     vector<int> parent, size;
3
4     DSU(int n) {
5         parent.resize(n);
6         size.assign(n, 1);
7         for (int i = 0; i < n; i++)
8             parent[i] = i;
9     }
10
11     int find(int x) {
12         if (parent[x] != x)
13             parent[x] = find(parent[x]); // Compresion de caminos
14         return parent[x];
15     }
16
17     bool unite(int x, int y) {
18         x = find(x);
19         y = find(y);
20         if (x == y) return false;
21         if (size[x] < size[y]) swap(x, y); // Union por tamaño
22         parent[y] = x;
23         size[x] += size[y];
24         return true;
25     }
26
27     bool same(int x, int y) {
28         return find(x) == find(y);
29     }
30
31     int componentSize(int x) {
32         return size[find(x)];
33     }
34 };
```

4.5. Variantes y extensiones

- **Unión por rango:** alternativa a la unión por tamaño, usando un arreglo `rank` para mantener la profundidad del árbol.
- **Contador de componentes:** se puede mantener una variable `components` que se decrementa cada vez que se realiza una unión exitosa.
- **DSU persistente:** permite retroceder en el tiempo (útil en consultas offline).
- **DSU con rollback:** guarda historial de cambios para revertir operaciones (útil en técnicas de divide & conquer).

4.6. Ejemplo mínimo de uso

```
1 int main() {
2     DSU dsu(5);
3     dsu.unite(0, 1);
4     dsu.unite(3, 4);
5     cout << dsu.same(0, 1) << endl; // Imprime 1
6     cout << dsu.same(1, 2) << endl; // Imprime 0
7 }
```

5. Segment Tree

5.1. Descripción general

El Segment Tree es una estructura de datos que permite responder consultas y realizar actualizaciones sobre intervalos de un arreglo de manera eficiente. Es especialmente útil cuando se requieren operaciones de rango (suma, mínimo, máximo, gcd, etc.) y actualizaciones frecuentes.

5.2. Complejidad

- Construcción: $\mathcal{O}(n)$
- Consulta de rango: $\mathcal{O}(\log n)$
- Actualización puntual o de rango: $\mathcal{O}(\log n)$

5.3. Implementación básica (suma de rangos)

```
1 struct SegmentTree {
2     int n;
3     vector<long long> tree;
4
5     SegmentTree(int n) {
```

```

6         this->n = n;
7         tree.assign(4*n, 0);
8     }
9
10    void build(vector<int>& arr, int node, int l, int r) {
11        if (l == r) {
12            tree[node] = arr[l];
13            return;
14        }
15        int mid = (l + r) / 2;
16        build(arr, 2*node, l, mid);
17        build(arr, 2*node+1, mid+1, r);
18        tree[node] = tree[2*node] + tree[2*node+1];
19    }
20
21    long long query(int node, int l, int r, int ql, int qr) {
22        if (qr < l || ql > r) return 0; // no overlap
23        if (ql <= l && r <= qr) return tree[node]; // total
24        overlap
25        int mid = (l + r) / 2;
26        return query(2*node, l, mid, ql, qr) +
27               query(2*node+1, mid+1, r, ql, qr);
28    }
29
30    void update(int node, int l, int r, int idx, int val) {
31        if (l == r) {
32            tree[node] = val;
33            return;
34        }
35        int mid = (l + r) / 2;
36        if (idx <= mid) update(2*node, l, mid, idx, val);
37        else update(2*node+1, mid+1, r, idx, val);
38        tree[node] = tree[2*node] + tree[2*node+1];
39    }
};

```

5.4. Segment Tree con Lazy Propagation

Cuando se requieren actualizaciones de rango (ejemplo: sumar un valor a todos los elementos de un intervalo), se utiliza lazy propagation para diferir actualizaciones y mantener eficiencia.

```

1 struct LazySegmentTree {
2     int n;
3     vector<long long> tree, lazy;
4
5     LazySegmentTree(int n) {
6         this->n = n;
7         tree.assign(4*n, 0);

```

```

8         lazy.assign(4*n, 0);
9     }
10
11     void push(int node, int l, int r) {
12         if (lazy[node] != 0) {
13             tree[node] += (r - l + 1) * lazy[node]; // apply
14                 update
15             if (l != r) {
16                 lazy[2*node] += lazy[node];
17                 lazy[2*node+1] += lazy[node];
18             }
19             lazy[node] = 0;
20         }
21     }
22
23     void updateRange(int node, int l, int r, int ql, int qr, int
24         val) {
25         push(node, l, r);
26         if (qr < l || ql > r) return; // no overlap
27         if (ql <= l && r <= qr) {
28             lazy[node] += val;
29             push(node, l, r);
30             return;
31         }
32         int mid = (l + r) / 2;
33         updateRange(2*node, l, mid, ql, qr, val);
34         updateRange(2*node+1, mid+1, r, ql, qr, val);
35         tree[node] = tree[2*node] + tree[2*node+1];
36     }
37
38     long long queryRange(int node, int l, int r, int ql, int qr) {
39         push(node, l, r);
40         if (qr < l || ql > r) return 0;
41         if (ql <= l && r <= qr) return tree[node];
42         int mid = (l + r) / 2;
43         return queryRange(2*node, l, mid, ql, qr) +
44             queryRange(2*node+1, mid+1, r, ql, qr);
45     }
46 };

```

5.5. Transformaciones y variantes

- **Segment Tree para mínimo/máximo:** cambiar la operación de combinación (min, max).
- **Segment Tree para gcd/lcm:** útil en problemas de divisibilidad.
- **Segment Tree de rango con asignación:** en lugar de sumar, se asigna un valor a todo un rango (requiere modificar la propagación).

- **Segment Tree de rango con multiplicación y suma:** se pueden combinar operaciones lineales (ejemplo: $f(x) = ax + b$).
- **Segment Tree 2D:** para consultas en matrices, con complejidad $\mathcal{O}(\log^2 n)$.

5.6. Segment Tree Persistente

Un Segment Tree persistente permite mantener versiones históricas de la estructura después de cada actualización, sin perder acceso a versiones anteriores. Esto se logra creando nuevos nodos solo en el camino de actualización.

```

1 struct PersistentSegmentTree {
2     struct Node {
3         long long val;
4         Node* left;
5         Node* right;
6         Node(long long v=0, Node* l=nullptr, Node* r=nullptr)
7             : val(v), left(l), right(r) {}
8     };
9
10    int n;
11    vector<Node*> versions;
12
13    PersistentSegmentTree(int n) {
14        this->n = n;
15        versions.push_back(build(0, n-1));
16    }
17
18    Node* build(int l, int r) {
19        if (l == r) return new Node(0);
20        int mid = (l + r) / 2;
21        return new Node(0, build(l, mid), build(mid+1, r));
22    }
23
24    Node* update(Node* node, int l, int r, int idx, int val) {
25        if (l == r) return new Node(val);
26        int mid = (l + r) / 2;
27        if (idx <= mid)
28            return new Node(node->val, update(node->left, l, mid,
29                                                idx, val), node->right);
30        else
31            return new Node(node->val, node->left, update(node->
32                                                            right, mid+1, r, idx, val));
33    }
34
35    long long query(Node* node, int l, int r, int ql, int qr) {
36        if (qr < l || ql > r) return 0;
37        if (ql <= l && r <= qr) return node->val;
38        int mid = (l + r) / 2;

```

```

37         return query(node->left, l, mid, ql, qr) +
38                query(node->right, mid+1, r, ql, qr);
39     }
40 };

```

5.7. Aplicaciones comunes

- Consultas de suma, mínimo, máximo en intervalos.
- Actualizaciones de rango con lazy propagation.
- Versionado de arreglos (persistencia).
- Problemas de geometría (rangos en 2D).
- Problemas de búsqueda binaria sobre respuestas con verificación en rangos.

6. Fenwick Tree (Binary Indexed Tree - BIT)

6.1. Descripción general

El Fenwick Tree (BIT) es una estructura de datos para mantener prefijos de un arreglo que soporta actualizaciones puntuales y consultas de prefijo en tiempo $\mathcal{O}(\log n)$. Es más simple y tiene menor memoria que un Segment Tree, aunque no soporta de forma directa algunas operaciones avanzadas sin transformaciones adicionales.

6.2. Complejidad

- Construcción desde ceros: $\mathcal{O}(n \log n)$ (o $\mathcal{O}(n)$ con técnica de *bulk build*).
- Actualización puntual: $\mathcal{O}(\log n)$.
- Consulta de prefijo: $\mathcal{O}(\log n)$.
- Consulta de rango: $\mathcal{O}(\log n)$ usando diferencias de prefijos.

6.3. BIT básico para suma (point update, prefix query)

```

1 struct BIT {
2     int n;
3     vector<long long> bit; // 1-indexed internal structure
4
5     BIT(int n) : n(n), bit(n + 1, 0) {}
6
7     void add(int idx, long long delta) {
8         // idx is 1-based
9         for (; idx <= n; idx += idx & -idx)
10             bit[idx] += delta;

```



```

11     }
12
13     long long sumPrefix(int idx) const {
14         long long res = 0;
15         for (; idx > 0; idx -= idx & -idx)
16             res += bit[idx];
17         return res;
18     }
19
20     long long sumRange(int l, int r) const {
21         if (l > r) return 0;
22         return sumPrefix(r) - sumPrefix(l - 1);
23     }
24
25     // Optional: build in O(n) if we have an array arr[1..n]
26     void build(const vector<long long>& arr) {
27         // arr must be 1-based aligned with size n
28         for (int i = 1; i <= n; i++) {
29             bit[i] += arr[i];
30             int j = i + (i & -i);
31             if (j <= n) bit[j] += bit[i];
32         }
33     }
34 };

```

6.4. BIT con compresión de coordenadas

En programación competitiva, los índices pueden ser valores grandes o arbitrarios. La compresión mapea valores a un rango $[1..m]$ ordenado.

```

1 struct Compressor {
2     vector<long long> vals;
3
4     void add(long long x) { vals.push_back(x); }
5
6     void build() {
7         sort(vals.begin(), vals.end());
8         vals.erase(unique(vals.begin(), vals.end()), vals.end());
9     }
10
11     int id(long long x) const {
12         int pos = int(lower_bound(vals.begin(), vals.end(), x) -
13                             vals.begin());
14         return pos + 1; // 1-based index for BIT
15     }
16
17     int size() const { return (int)vals.size(); }
18 };

```

6.5. Transformaciones estándar

6.5.1. Range query con point update

La estructura básica soporta `sumRange(l,r)` vía prefijos con `add(idx, delta)`.

6.5.2. Point query con range update

Usando un BIT sobre el arreglo de diferencias, podemos realizar actualizaciones de rango y obtener el valor puntual:

- Para actualizar rango $[l, r]$ con $+val$: `add(l, val), add(r+1, -val)`.
- Para consultar un punto `idx`: `sumPrefix(idx)`.

```
1 struct RangeUpdatePointQueryBIT {
2     int n;
3     BIT bit; // stores difference array updates
4
5     RangeUpdatePointQueryBIT(int n) : n(n), bit(n) {}
6
7     void rangeAdd(int l, int r, long long val) {
8         bit.add(l, val);
9         if (r + 1 <= n) bit.add(r + 1, -val);
10    }
11
12    long long pointQuery(int idx) const {
13        return bit.sumPrefix(idx);
14    }
15 };
```

6.5.3. Range update con range query

Usando dos BITs para mantener transformaciones lineales:

$$\text{sumPrefix}(x) = x \cdot B_1(x) - B_2(x)$$

donde se aplican actualizaciones:

`rangeAdd(l, r, val) :` `B1.add(l, val), B1.add(r+1, -val); B2.add(l, val·(l-1)), B2.add(r+1, -val·r)`

```
1 struct RangeUpdateRangeQueryBIT {
2     int n;
3     BIT B1, B2;
4
5     RangeUpdateRangeQueryBIT(int n) : n(n), B1(n), B2(n) {}
6
7     void internalAdd(BIT& B, int idx, long long val) { B.add(idx,
8         val); }
```

```

8
9 void rangeAdd(int l, int r, long long val) {
10     internalAdd(B1, l, val);
11     if (r + 1 <= n) internalAdd(B1, r + 1, -val);
12
13     internalAdd(B2, l, val * (l - 1));
14     if (r + 1 <= n) internalAdd(B2, r + 1, -val * r);
15 }
16
17 long long prefixSum(int x) const {
18     return x * B1.sumPrefix(x) - B2.sumPrefix(x);
19 }
20
21 long long rangeSum(int l, int r) const {
22     return prefixSum(r) - prefixSum(l - 1);
23 }
24 };

```

6.6. BIT para otras operaciones

6.6.1. BIT para XOR

La operación XOR es invertible y asociativa, se puede usar la misma lógica que suma:

```

1 struct BITXOR {
2     int n;
3     vector<int> bit;
4
5     BITXOR(int n) : n(n), bit(n + 1, 0) {}
6
7     void add(int idx, int val) {
8         for (; idx <= n; idx += idx & -idx)
9             bit[idx] ^= val; // xor instead of addition
10    }
11
12    int prefixXor(int idx) const {
13        int res = 0;
14        for (; idx > 0; idx -= idx & -idx)
15            res ^= bit[idx];
16        return res;
17    }
18
19    int rangeXor(int l, int r) const {
20        return prefixXor(r) ^ prefixXor(l - 1);
21    }
22 };

```

6.6.2. BIT para mínimo/máximo

Para mantener el mínimo de prefijo, se puede usar un BIT con operación `min`, pero las actualizaciones dejan de ser inversibles en el mismo sentido. Por lo general, se usa un *Fenwick tree* con `max` para *prefix maximum* y consultas adecuadas, teniendo cuidado con el modelo (no es simétrico como suma). En muchos casos, se prefiere Segment Tree para `min/max`.

```
1 struct BITMax {
2     int n;
3     vector<long long> bit;
4
5     BITMax(int n) : n(n), bit(n + 1, LLONG_MIN) {}
6
7     void update(int idx, long long val) {
8         for (; idx <= n; idx += idx & -idx)
9             bit[idx] = max(bit[idx], val);
10    }
11
12    long long prefixMax(int idx) const {
13        long long res = LLONG_MIN;
14        for (; idx > 0; idx -= idx & -idx)
15            res = max(res, bit[idx]);
16        return res;
17    }
18 };
```

6.7. K-ésimo elemento (order statistics) con BIT de frecuencias

Si el BIT almacena frecuencias (cuentas), podemos encontrar el menor índice `idx` tal que `sumPrefix(idx) >= k` en $\mathcal{O}(\log n)$ usando *binary lifting* dentro del árbol interno.

```
1 struct BITFreq {
2     int n;
3     vector<int> bit;
4
5     BITFreq(int n) : n(n), bit(n + 1, 0) {}
6
7     void add(int idx, int delta) {
8         for (; idx <= n; idx += idx & -idx)
9             bit[idx] += delta;
10    }
11
12    int sumPrefix(int idx) const {
13        int res = 0;
14        for (; idx > 0; idx -= idx & -idx)
15            res += bit[idx];
16        return res;
17    }
18 };
```

```

18
19 // Find smallest idx such that sumPrefix(idx) >= k
20 int kth(int k) const {
21     int idx = 0;
22     int mask = 1;
23     while (mask <= n) mask <<= 1; // highest power of two >= n
24     for (int step = mask; step > 0; step >>= 1) {
25         int next = idx + step;
26         if (next <= n && bit[next] < k) {
27             idx = next;
28             k -= bit[next];
29         }
30     }
31     return idx + 1;
32 }
33 };

```

6.8. Inversiones en un arreglo (uso clásico con compresión)

Contar inversiones: iterar elementos de derecha a izquierda, sumar cuántos ya vistos son menores/ mayores según criterio y actualizar frecuencia.

```

1 long long countInversions(vector<int> a) {
2     Compressor comp;
3     for (int x : a) comp.add(x);
4     comp.build();
5     int m = comp.size();
6     BITFreq bit(m);
7     long long inv = 0;
8     for (int i = (int)a.size() - 1; i >= 0; --i) {
9         int id = comp.id(a[i]);
10        inv += bit.sumPrefix(id - 1); // count elements < a[i]
11        // already seen
12        bit.add(id, 1);
13    }
14    return inv;
15 }

```

6.9. BIT 2D

Para matrices 2D, se puede usar un BIT bidimensional con complejidad $\mathcal{O}(\log^2 n)$.

```

1 struct BIT2D {
2     int n, m;
3     vector<vector<long long>> bit;
4
5     BIT2D(int n, int m) : n(n), m(m), bit(n + 1, vector<long long>
6         >(m + 1, 0)) {}

```

```

7     void add(int x, int y, long long delta) {
8         for (int i = x; i <= n; i += i & -i)
9             for (int j = y; j <= m; j += j & -j)
10                bit[i][j] += delta;
11     }
12
13     long long sumPrefix(int x, int y) const {
14         long long res = 0;
15         for (int i = x; i > 0; i -= i & -i)
16             for (int j = y; j > 0; j -= j & -j)
17                 res += bit[i][j];
18         return res;
19     }
20
21     long long sumRect(int x1, int y1, int x2, int y2) const {
22         return sumPrefix(x2, y2) - sumPrefix(x1 - 1, y2)
23            - sumPrefix(x2, y1 - 1) + sumPrefix(x1 - 1, y1 - 1);
24     }
25 };

```

7. Sparse Table (RMQ)

7.1. Descripción general

La Sparse Table es una estructura de datos estática que permite responder consultas de rango en tiempo $\mathcal{O}(1)$ después de una precomputación en $\mathcal{O}(n \log n)$. Es ideal cuando el arreglo no cambia (sin actualizaciones) y se requieren muchas consultas de rango.

7.2. Complejidad

- Preprocesamiento: $\mathcal{O}(n \log n)$
- Consulta de rango: $\mathcal{O}(1)$
- Actualizaciones: No soportadas (estructura estática)

7.3. Idea principal

Se precomputan respuestas para todos los intervalos de longitud 2^k para cada k . Para responder una consulta sobre un rango $[L, R]$, se toma $k = \lfloor \log_2(R - L + 1) \rfloor$ y se combinan las dos tablas precomputadas que cubren el rango.

7.4. Implementación para RMQ (mínimo en rango)

```

1 struct SparseTable {
2     int n, K;
3     vector<vector<int>> st;

```

```

4     vector<int> log2table;
5
6     SparseTable(const vector<int>& arr) {
7         n = (int)arr.size();
8         K = 32 - __builtin_clz(n); // max power of two needed
9         st.assign(n, vector<int>(K));
10        log2table.resize(n + 1);
11
12        // Precompute logs
13        log2table[1] = 0;
14        for (int i = 2; i <= n; i++)
15            log2table[i] = log2table[i/2] + 1;
16
17        // Initialize st for intervals of length 1
18        for (int i = 0; i < n; i++)
19            st[i][0] = arr[i];
20
21        // Compute values for larger intervals
22        for (int j = 1; j < K; j++) {
23            for (int i = 0; i + (1 << j) <= n; i++) {
24                st[i][j] = min(st[i][j-1], st[i + (1 << (j-1))][j-1]);
25            }
26        }
27    }
28
29    int query(int L, int R) {
30        int j = log2table[R - L + 1];
31        return min(st[L][j], st[R - (1 << j) + 1][j]);
32    }
33 };

```

7.5. Variantes

- **RMQ (Range Minimum Query):** clásico, como el ejemplo.
- **RMaxQ (Range Maximum Query):** cambiar la operación a max.
- **GCD en rangos:** usar gcd como operación de combinación.
- **Operaciones idempotentes:** Sparse Table funciona para operaciones idempotentes y asociativas (ejemplo: min, max, gcd, lcm).
- **No funciona para suma:** no es idempotente, por lo que no se puede responder en $\mathcal{O}(1)$ con Sparse Table.

7.6. Aplicaciones en programación competitiva

- Consultas de mínimo/máximo en arreglos estáticos.

- Preprocesamiento para algoritmos de LCA (Lowest Common Ancestor) en árboles mediante Euler Tour + RMQ.
- Consultas de gcd en rangos para problemas de divisibilidad.
- Verificación de condiciones en subarreglos estáticos.

7.7. Ejemplo de uso

```

1 int main() {
2     vector<int> arr = {1, 3, -2, 8, -7, 10};
3     SparseTable st(arr);
4     cout << st.query(1, 3) << endl; // min in [1,3] = -2
5     cout << st.query(2, 4) << endl; // min in [2,4] = -7
6 }

```

8. Hashing de Strings

8.1. Descripción general

El hashing de strings es una técnica que permite representar una cadena como un número entero de forma que se puedan comparar substrings en tiempo constante después de un preprocesamiento. Se basa en funciones de hash polinómicas y en el uso de aritmética modular. Es ampliamente utilizado en programación competitiva para:

- Comparación rápida de substrings.
- Búsqueda de patrones en cadenas.
- Detección de palíndromos.
- Construcción de estructuras avanzadas como suffix arrays o automaton.

8.2. Hash polinómico (rolling hash)

La idea es representar un string $S = s_0s_1 \dots s_{n-1}$ como:

$$H(S) = \sum_{i=0}^{n-1} (s_i \cdot p^i) \quad \text{mód } M$$

donde p es una base (número primo cercano al tamaño del alfabeto) y M es un número primo grande.

8.3. Implementación de rolling hash

```
1 struct RollingHash {
2     int n;
3     long long base, mod;
4     vector<long long> prefix, power;
5
6     RollingHash(const string& s, long long base=911382323, long
7         long mod=1000000007) {
8         this->n = (int)s.size();
9         this->base = base;
10        this->mod = mod;
11        prefix.assign(n+1, 0);
12        power.assign(n+1, 1);
13
14        for (int i = 0; i < n; i++) {
15            prefix[i+1] = (prefix[i] * base + s[i]) % mod;
16            power[i+1] = (power[i] * base) % mod;
17        }
18
19        long long getHash(int l, int r) {
20            // substring [l, r]
21            long long res = (prefix[r+1] - (prefix[l] * power[r-l+1])
22                % mod + mod) % mod;
23            return res;
24        }
25    };
26 }
```

8.4. Double hashing

Para reducir la probabilidad de colisiones, se utilizan dos funciones de hash con diferentes módulos y bases. La comparación de substrings se hace verificando ambos hashes.

```
1 struct DoubleHash {
2     RollingHash h1, h2;
3
4     DoubleHash(const string& s)
5         : h1(s, 911382323, 1000000007),
6           h2(s, 972663749, 1000000009) {}
7
8     pair<long long, long long> getHash(int l, int r) {
9         return {h1.getHash(l, r), h2.getHash(l, r)};
10    }
11 };
12 }
```

8.5. Aplicaciones comunes

- **Comparación de substrings:** verificar igualdad en $\mathcal{O}(1)$.
- **Búsqueda de patrones (Rabin-Karp):** comparar hash del patrón con substrings de la cadena.
- **Detección de palíndromos:** mantener hash directo e inverso.
- **Verificación de rotaciones:** concatenar la cadena consigo misma y buscar el patrón.
- **Preprocesamiento para estructuras avanzadas:** suffix array, suffix automaton.

8.6. Hash bidireccional (forward y backward)

Se puede mantener un hash de izquierda a derecha y otro de derecha a izquierda para verificar palíndromos en tiempo constante.

```
1 struct PalindromeHash {
2     RollingHash fwd, bwd;
3     string s;
4     int n;
5
6     PalindromeHash(const string& s)
7         : s(s), n((int)s.size()),
8           fwd(s, 911382323, 1000000007) {
9         string rs = s;
10        reverse(rs.begin(), rs.end());
11        bwd = RollingHash(rs, 911382323, 1000000007);
12    }
13
14    bool isPalindrome(int l, int r) {
15        long long h1 = fwd.getHash(l, r);
16        long long h2 = bwd.getHash(n-1-r, n-1-l);
17        return h1 == h2;
18    }
19 };
```

8.7. Consideraciones avanzadas

- **Elección de base y módulo:** la base debe ser mayor al tamaño del alfabeto, y el módulo debe ser un número primo grande.
- **Colisiones:** aunque poco probables, pueden ocurrir. El double hashing reduce drásticamente el riesgo.
- **Overflow:** usar unsigned long long o aritmética modular explícita para evitar errores.

- **Hashing con FFT/NTT:** en algunos problemas se puede usar convolución para comparar substrings o patrones.

8.8. Ejemplo de uso

```

1 int main() {
2     string s = "abacaba";
3     DoubleHash dh(s);
4     cout << (dh.getHash(0,2) == dh.getHash(4,6)) << endl; // "aba"
5     }

```

9. Trie y Aho-Corasick

9.1. Descripción general

El **Trie** es una estructura de datos en forma de árbol que permite almacenar y buscar cadenas de manera eficiente, caracterizada por compartir prefijos comunes. El algoritmo **Aho-Corasick** es una extensión del Trie que permite realizar búsqueda de múltiples patrones en un texto en tiempo lineal respecto al tamaño del texto más el número de ocurrencias encontradas.

9.2. Complejidad

- Inserción en Trie: $\mathcal{O}(L)$, donde L es la longitud de la cadena.
- Búsqueda en Trie: $\mathcal{O}(L)$.
- Construcción de Aho-Corasick: $\mathcal{O}(\sum L_i + \Sigma)$, donde $\sum L_i$ es la suma de longitudes de los patrones y Σ es el tamaño del alfabeto.
- Búsqueda en Aho-Corasick: $\mathcal{O}(N + Z)$, donde N es la longitud del texto y Z el número de ocurrencias.

9.3. Implementación básica de Trie

```

1 struct TrieNode {
2     array<int, 26> next;
3     bool isEnd;
4     TrieNode() {
5         next.fill(-1);
6         isEnd = false;
7     }
8 };
9
10 struct Trie {

```

```

11     vector<TrieNode> nodes;
12     Trie() { nodes.push_back(TrieNode()); }
13
14     void insert(const string& s) {
15         int cur = 0;
16         for (char c : s) {
17             int idx = c - 'a';
18             if (nodes[cur].next[idx] == -1) {
19                 nodes[cur].next[idx] = (int)nodes.size();
20                 nodes.push_back(TrieNode());
21             }
22             cur = nodes[cur].next[idx];
23         }
24         nodes[cur].isEnd = true;
25     }
26
27     bool search(const string& s) {
28         int cur = 0;
29         for (char c : s) {
30             int idx = c - 'a';
31             if (nodes[cur].next[idx] == -1) return false;
32             cur = nodes[cur].next[idx];
33         }
34         return nodes[cur].isEnd;
35     }
36 };

```

9.4. Aho-Corasick

El algoritmo Aho-Corasick construye un automáta a partir de un Trie de patrones, añadiendo enlaces de fallo (*failure links*) que permiten saltar a otros nodos cuando no hay coincidencia, similar al algoritmo KMP pero extendido a múltiples patrones.

```

1 struct AhoCorasick {
2     struct Node {
3         array<int, 26> next;
4         int link; // failure link
5         vector<int> out; // indices of patterns ending here
6         Node() {
7             next.fill(-1);
8             link = -1;
9         }
10    };
11
12    vector<Node> trie;
13
14    AhoCorasick() { trie.push_back(Node()); }
15
16    void insert(const string& s, int id) {

```

```

17     int cur = 0;
18     for (char c : s) {
19         int idx = c - 'a';
20         if (trie[cur].next[idx] == -1) {
21             trie[cur].next[idx] = (int)trie.size();
22             trie.push_back(Node());
23         }
24         cur = trie[cur].next[idx];
25     }
26     trie[cur].out.push_back(id);
27 }
28
29 void build() {
30     queue<int> q;
31     trie[0].link = 0;
32     for (int c = 0; c < 26; c++) {
33         int nxt = trie[0].next[c];
34         if (nxt != -1) {
35             trie[nxt].link = 0;
36             q.push(nxt);
37         } else {
38             trie[0].next[c] = 0;
39         }
40     }
41     while (!q.empty()) {
42         int v = q.front(); q.pop();
43         for (int c = 0; c < 26; c++) {
44             int nxt = trie[v].next[c];
45             if (nxt != -1) {
46                 trie[nxt].link = trie[trie[v].link].next[c];
47                 for (int id : trie[trie[nxt].link].out)
48                     trie[nxt].out.push_back(id);
49                 q.push(nxt);
50             } else {
51                 trie[v].next[c] = trie[trie[v].link].next[c];
52             }
53         }
54     }
55 }
56
57 vector<pair<int,int>> search(const string& text) {
58     vector<pair<int,int>> res;
59     int cur = 0;
60     for (int i = 0; i < (int)text.size(); i++) {
61         int c = text[i] - 'a';
62         cur = trie[cur].next[c];
63         for (int id : trie[cur].out) {
64             res.push_back({i, id});
65             // pattern id found ending at position i

```

```

66         }
67     }
68     return res;
69 }
70 };

```

9.5. Aplicaciones comunes

- Búsqueda de múltiples patrones en un texto (ejemplo: detectar palabras prohibidas).
- Problemas de coincidencia de cadenas en línea.
- Bioinformática: búsqueda de secuencias de ADN.
- Compresión de datos y detección de substrings frecuentes.

9.6. Variantes y extensiones

- **Trie comprimido (Radix Tree)**: optimiza memoria al comprimir cadenas de nodos con un solo hijo.
- **Trie con punteros dinámicos**: útil para alfabetos grandes o no conocidos.
- **Aho-Corasick con pesos**: cada patrón tiene un valor asociado, se acumulan en las ocurrencias.
- **Automatización de reemplazo**: reemplazar patrones encontrados en un texto en tiempo lineal.

9.7. Ejemplo de uso

```

1 int main() {
2     vector<string> patterns = {"he", "she", "his", "hers"};
3     AhoCorasick ac;
4     for (int i = 0; i < (int)patterns.size(); i++)
5         ac.insert(patterns[i], i);
6     ac.build();
7     string text = "ahishers";
8     auto matches = ac.search(text);
9     for (auto [pos, id] : matches) {
10         cout << "Pattern " << patterns[id]
11             << " found ending at index " << pos << endl;
12     }
13 }

```

10. BFS y DFS

10.1. Descripción general

Los algoritmos **BFS** (**Breadth-First Search**) y **DFS** (**Depth-First Search**) son fundamentales en el análisis de grafos. Permiten recorrer todos los nodos y aristas de un grafo, y son la base de múltiples algoritmos más avanzados.

10.2. Complejidad

- Tiempo: $\mathcal{O}(n + m)$, donde n es el número de vértices y m el número de aristas.
- Espacio: $\mathcal{O}(n)$ para estructuras auxiliares (visitados, colas o pilas).

10.3. Implementación de BFS

El BFS recorre el grafo por niveles, utilizando una cola. Es útil para encontrar distancias mínimas en grafos no ponderados.

```
1 vector<vector<int>>> adj;  
2 vector<int> dist;  
3  
4 void bfs(int start) {  
5     int n = (int)adj.size();  
6     dist.assign(n, -1);  
7     queue<int> q;  
8     dist[start] = 0;  
9     q.push(start);  
10    while (!q.empty()) {  
11        int u = q.front(); q.pop();  
12        for (int v : adj[u]) {  
13            if (dist[v] == -1) {  
14                dist[v] = dist[u] + 1;  
15                q.push(v);  
16            }  
17        }  
18    }  
19 }
```

10.4. Implementación de DFS

El DFS recorre el grafo en profundidad, utilizando recursión o una pila explícita. Es útil para detectar ciclos, componentes conexas y ordenamientos topológicos.

```
1 vector<vector<int>>> adj;  
2 vector<bool> visited;  
3  
4 void dfs(int u) {  
5     visited[u] = true;
```

```

6     for (int v : adj[u]) {
7         if (!visited[v]) {
8             dfs(v);
9         }
10    }
11 }

```

10.5. Variantes de DFS

- **DFS iterativo:** evita problemas de stack overflow en grafos grandes.
- **DFS con tiempos de entrada y salida:** útil para algoritmos de árboles, detección de ciclos y análisis de puentes/articulation points.
- **DFS en árboles:** permite calcular tamaños de subárboles, DP en árboles y LCA.

10.6. Aplicaciones comunes de BFS

- Cálculo de distancias mínimas en grafos no ponderados.
- Verificación de bipartición de un grafo.
- Construcción de árboles de expansión mínima en grafos no ponderados.
- Algoritmos de flujo (nivelación en Dinic).

10.7. Aplicaciones comunes de DFS

- Detección de ciclos en grafos dirigidos y no dirigidos.
- Cálculo de componentes conexas.
- Ordenamiento topológico.
- Algoritmos de SCC (Kosaraju, Tarjan).
- Detección de puentes y articulation points.

10.8. Ejemplo de uso combinado

```

1 int main() {
2     int n = 6;
3     adj.assign(n, {});
4     adj[0] = {1, 2};
5     adj[1] = {0, 3};
6     adj[2] = {0, 4};
7     adj[3] = {1, 5};
8     adj[4] = {2};
9     adj[5] = {3};

```



```

10
11 // BFS desde nodo 0
12 bfs(0);
13 for (int i = 0; i < n; i++)
14     cout << "Distancia a " << i << " = " << dist[i] << endl;
15
16 // DFS desde nodo 0
17 visited.assign(n, false);
18 dfs(0);
19 for (int i = 0; i < n; i++)
20     cout << "Visitado " << i << " = " << visited[i] << endl;
21 }

```

11. Algoritmos de Caminos Mínimos: Dijkstra, Bellman-Ford y Floyd-Warshall

11.1. Descripción general

Los algoritmos de caminos mínimos permiten calcular la distancia más corta entre nodos en un grafo ponderado. Dependiendo de las características del grafo (pesos positivos, negativos, densidad), se utilizan diferentes algoritmos:

- **Dijkstra:** grafos con pesos no negativos.
- **Bellman-Ford:** grafos con pesos negativos, detecta ciclos negativos.
- **Floyd-Warshall:** todos los pares de nodos, soporta pesos negativos (sin ciclos negativos).

11.2. Dijkstra

11.2.1. Complejidad

- Con cola de prioridad (heap): $\mathcal{O}((n + m) \log n)$.
- Con matriz de adyacencia: $\mathcal{O}(n^2)$.

11.2.2. Implementación

```

1 struct Edge { int to; long long w; };
2 vector<vector<Edge>> adj;
3 vector<long long> dist;
4
5 void dijkstra(int start) {
6     int n = (int)adj.size();
7     dist.assign(n, LLONG_MAX);
8     dist[start] = 0;

```

```

9     priority_queue<pair<long long,int>, vector<pair<long long,int
    >>, greater<>> pq;
10    pq.push({0, start});
11    while (!pq.empty()) {
12        auto [d,u] = pq.top(); pq.pop();
13        if (d != dist[u]) continue;
14        for (auto e : adj[u]) {
15            if (dist[e.to] > d + e.w) {
16                dist[e.to] = d + e.w;
17                pq.push({dist[e.to], e.to});
18            }
19        }
20    }
21 }

```

11.2.3. Aplicaciones

- Caminos mínimos en grafos con pesos positivos.
- Redes de transporte, enrutamiento en redes.
- Base para algoritmos de flujo (Dinic con scaling).

11.3. Bellman-Ford

11.3.1. Complejidad

- Tiempo: $\mathcal{O}(n \cdot m)$.
- Espacio: $\mathcal{O}(n)$.

11.3.2. Implementación

```

1 struct Edge { int u, v; long long w; };
2 vector<Edge> edges;
3 vector<long long> dist;
4
5 bool bellmanFord(int n, int start) {
6     dist.assign(n, LLONG_MAX);
7     dist[start] = 0;
8     for (int i = 0; i < n-1; i++) {
9         for (auto e : edges) {
10             if (dist[e.u] != LLONG_MAX && dist[e.u] + e.w < dist[e
                .v]) {
11                 dist[e.v] = dist[e.u] + e.w;
12             }
13         }
14     }

```

```

15     // Detect negative cycle
16     for (auto e : edges) {
17         if (dist[e.u] != LLONG_MAX && dist[e.u] + e.w < dist[e.v])
18             {
19                 return false; // negative cycle detected
20             }
21     }
22     return true;
23 }

```

11.3.3. Aplicaciones

- Grafos con pesos negativos.
- Detección de ciclos negativos.
- Problemas de optimización con restricciones lineales.

11.4. Floyd-Warshall

11.4.1. Complejidad

- Tiempo: $\mathcal{O}(n^3)$.
- Espacio: $\mathcal{O}(n^2)$.

11.4.2. Implementación

```

1 vector<vector<long long>> dist;
2
3 void floydWarshall(int n) {
4     for (int k = 0; k < n; k++) {
5         for (int i = 0; i < n; i++) {
6             for (int j = 0; j < n; j++) {
7                 if (dist[i][k] < LLONG_MAX && dist[k][j] <
8                     LLONG_MAX)
9                     dist[i][j] = min(dist[i][j], dist[i][k] + dist
10                                     [k][j]);
11             }
12         }
13     }
14 }

```

11.4.3. Aplicaciones

- Caminos mínimos entre todos los pares de nodos.
- Detección de ciclos negativos (si $dist[i][i] < 0$).

- Problemas de cierre transitivo y análisis de grafos densos.

11.5. Comparación

- **Dijkstra**: rápido, pero solo para pesos no negativos.
- **Bellman-Ford**: más lento, pero soporta pesos negativos y detecta ciclos negativos.
- **Floyd-Warshall**: todos los pares, útil en grafos densos y cuando n es pequeño (hasta ~ 500).

12. Árbol de Expansión Mínima: Kruskal y Prim

12.1. Descripción general

Un **árbol de expansión mínima** (MST, **Minimum Spanning Tree**) de un grafo no dirigido y conexo es un subconjunto de aristas que conecta todos los vértices sin ciclos y con el peso total mínimo posible. Los algoritmos más utilizados para construir un MST son **Kruskal** y **Prim**.

12.2. Algoritmo de Kruskal

12.2.1. Idea principal

Ordenar todas las aristas por peso e ir añadiéndolas al MST si no forman un ciclo, utilizando una estructura **Union-Find** (DSU) para verificar la conectividad.

12.2.2. Complejidad

- Ordenamiento de aristas: $\mathcal{O}(m \log m)$.
- Operaciones de DSU: $\mathcal{O}(m\alpha(n))$.
- Complejidad total: $\mathcal{O}(m \log m)$.

12.2.3. Implementación

```

1 struct Edge {
2     int u, v;
3     long long w;
4     bool operator< (const Edge& other) const {
5         return w < other.w;
6     }
7 };
8
9 struct DSU {
10     vector<int> parent, sz;
11     DSU(int n) {

```

```

12         parent.resize(n);
13         sz.assign(n, 1);
14         for (int i = 0; i < n; i++) parent[i] = i;
15     }
16     int find(int x) {
17         return parent[x] == x ? x : parent[x] = find(parent[x]);
18     }
19     bool unite(int a, int b) {
20         a = find(a); b = find(b);
21         if (a == b) return false;
22         if (sz[a] < sz[b]) swap(a, b);
23         parent[b] = a;
24         sz[a] += sz[b];
25         return true;
26     }
27 };
28
29 long long kruskal(int n, vector<Edge>& edges) {
30     sort(edges.begin(), edges.end());
31     DSU dsu(n);
32     long long mst_cost = 0;
33     for (auto& e : edges) {
34         if (dsu.unite(e.u, e.v)) {
35             mst_cost += e.w;
36         }
37     }
38     return mst_cost;
39 }

```

12.2.4. Aplicaciones

- Construcción de MST en grafos dispersos.
- Problemas de clustering (agrupamiento jerárquico).
- Diseño de redes de bajo costo.

12.3. Algoritmo de Prim

12.3.1. Idea principal

Construir el MST de manera incremental, comenzando desde un nodo y añadiendo siempre la arista más barata que conecta un vértice dentro del MST con uno fuera de él. Se implementa eficientemente con una cola de prioridad.

12.3.2. Complejidad

- Con cola de prioridad binaria: $\mathcal{O}(m \log n)$.

- Con Fibonacci heap: $\mathcal{O}(m + n \log n)$ (teórico, rara vez usado en práctica).

12.3.3. Implementación

```

1 struct Edge { int to; long long w; };
2 vector<vector<Edge>> adj;
3
4 long long prim(int n) {
5     vector<bool> inMST(n, false);
6     vector<long long> minEdge(n, LLONG_MAX);
7     priority_queue<pair<long long, int>, vector<pair<long long, int>
8         >>, greater<>> pq;
9
10    minEdge[0] = 0;
11    pq.push({0, 0});
12    long long mst_cost = 0;
13
14    while (!pq.empty()) {
15        auto [w,u] = pq.top(); pq.pop();
16        if (inMST[u]) continue;
17        inMST[u] = true;
18        mst_cost += w;
19        for (auto e : adj[u]) {
20            if (!inMST[e.to] && e.w < minEdge[e.to]) {
21                minEdge[e.to] = e.w;
22                pq.push({e.w, e.to});
23            }
24        }
25    }
26    return mst_cost;
27 }
```

12.3.4. Aplicaciones

- Construcción de MST en grafos densos.
- Problemas de diseño de redes donde se parte de un nodo inicial.
- Casos donde se requiere construir el MST en línea (añadiendo nodos progresivamente).

12.4. Comparación entre Kruskal y Prim

- **Kruskal**: más eficiente en grafos dispersos, fácil de implementar con DSU.
- **Prim**: más eficiente en grafos densos, especialmente con estructuras de prioridad avanzadas.

- Ambos producen el mismo MST, pero su eficiencia depende de la densidad del grafo.

13. Longest Increasing Subsequence (LIS)

13.1. Descripción general

El problema de la **subsecuencia creciente más larga (LIS, Longest Increasing Subsequence)** consiste en encontrar la subsecuencia más larga de un arreglo tal que todos sus elementos estén en orden estrictamente creciente. Es un problema clásico de programación dinámica y tiene múltiples aplicaciones en análisis de secuencias, bioinformática y optimización.

13.2. Complejidad

- DP ingenuo: $\mathcal{O}(n^2)$.
- Optimizado con búsqueda binaria: $\mathcal{O}(n \log n)$.

13.3. Implementación en $\mathcal{O}(n^2)$

```

1 int LISn2(vector<int>& a) {
2     int n = (int)a.size();
3     vector<int> dp(n, 1);
4     int ans = 0;
5     for (int i = 0; i < n; i++) {
6         for (int j = 0; j < i; j++) {
7             if (a[j] < a[i]) {
8                 dp[i] = max(dp[i], dp[j] + 1);
9             }
10        }
11        ans = max(ans, dp[i]);
12    }
13    return ans;
14 }
```

13.4. Implementación en $\mathcal{O}(n \log n)$

La idea es mantener un arreglo `tail` donde `tail[k]` es el menor valor posible del último elemento de una subsecuencia creciente de longitud $k + 1$. Se actualiza usando `lower_bound`.

```

1 int LISnlogn(vector<int>& a) {
2     vector<int> tail;
3     for (int x : a) {
4         auto it = lower_bound(tail.begin(), tail.end(), x);
5         if (it == tail.end()) tail.push_back(x);
6     }
```

```

6         else *it = x;
7     }
8     return (int)tail.size();
9 }

```

13.5. Reconstrucción de la subsecuencia

Además de calcular la longitud, se puede reconstruir la subsecuencia manteniendo predecesores.

```

1 vector<int> LISsequence(vector<int>& a) {
2     int n = (int)a.size();
3     vector<int> tail, tailIndex, prev(n, -1);
4     for (int i = 0; i < n; i++) {
5         auto it = lower_bound(tail.begin(), tail.end(), a[i]);
6         int idx = (int)(it - tail.begin());
7         if (it == tail.end()) {
8             tail.push_back(a[i]);
9             tailIndex.push_back(i);
10        } else {
11            *it = a[i];
12            tailIndex[idx] = i;
13        }
14        if (idx > 0) prev[i] = tailIndex[idx-1];
15    }
16    vector<int> lis;
17    for (int i = tailIndex.back(); i >= 0; i = prev[i])
18        lis.push_back(a[i]);
19    reverse(lis.begin(), lis.end());
20    return lis;
21 }

```

13.6. Aplicaciones comunes

- Análisis de secuencias en bioinformática.
- Problemas de ordenamiento parcial.
- Optimización en cadenas de producción.
- Subproblema en DP multidimensional (ejemplo: LIS en 2D con pares).

13.7. Variantes avanzadas

- **LDS (Longest Decreasing Subsequence)**: basta con invertir el criterio de comparación.
- **LIS en 2D**: ordenar por una dimensión y aplicar LIS en la otra (ejemplo: problema de los sobres).

- **LIS en línea:** mantener estructura de datos (árbol segmentado o BIT) para procesar secuencias en streaming.
- **LIS con restricciones:** subsecuencia creciente con saltos limitados o condiciones adicionales.

13.8. Ejemplo de uso

```

1 int main() {
2     vector<int> a = {10, 9, 2, 5, 3, 7, 101, 18};
3     cout << "LIS O(n^2): " << LISn2(a) << endl;
4     cout << "LIS O(n log n): " << LISnlogn(a) << endl;
5     auto seq = LISsequence(a);
6     cout << "LIS sequence: ";
7     for (int x : seq) cout << x << " ";
8     cout << endl;
9 }

```

14. DP con Bitmasks

14.1. Descripción general

La técnica de **DP con bitmasks** se utiliza cuando el estado de la DP depende de un subconjunto de elementos. En lugar de representar el subconjunto con una estructura compleja, se usa un entero donde el bit i indica si el elemento i está incluido o no.

14.2. Complejidad

- Número de estados: 2^n (todos los subconjuntos).
- Transiciones: depende del problema, típicamente $\mathcal{O}(n \cdot 2^n)$.
- Factible para $n \leq 20$ (aprox).

14.3. Ejemplo clásico: Traveling Salesman Problem (TSP)

Dado un grafo completo con n nodos y matriz de distancias $dist[i][j]$, encontrar el ciclo Hamiltoniano de costo mínimo.

14.3.1. Definición de estado

$dp[mask][u]$ = costo mínimo para visitar el subconjunto de nodos en $mask$ y terminar en u

14.3.2. Recurrencia

$$dp[mask][u] = \min_{v \in mask, v \neq u} (dp[mask \setminus \{u\}][v] + dist[v][u])$$

14.3.3. Implementación

```
1 const long long INF = 1e15;
2
3 long long tsp(int n, vector<vector<long long>>& dist) {
4     int N = 1 << n;
5     vector<vector<long long>> dp(N, vector<long long>(n, INF));
6     dp[1][0] = 0; // start at node 0
7
8     for (int mask = 1; mask < N; mask++) {
9         for (int u = 0; u < n; u++) {
10             if (!(mask & (1<<u))) continue;
11             for (int v = 0; v < n; v++) {
12                 if (mask & (1<<v)) continue;
13                 int nextMask = mask | (1<<v);
14                 dp[nextMask][v] = min(dp[nextMask][v],
15                                     dp[mask][u] + dist[u][v]);
16             }
17         }
18     }
19
20     long long ans = INF;
21     for (int u = 0; u < n; u++) {
22         ans = min(ans, dp[(1<<n)-1][u] + dist[u][0]);
23     }
24     return ans;
25 }
```

14.4. Ejemplo: Subset Sum con bitmask DP

```
1 bool subsetSum(vector<int>& a, int target) {
2     int n = (int)a.size();
3     int N = 1 << n;
4     for (int mask = 0; mask < N; mask++) {
5         int sum = 0;
6         for (int i = 0; i < n; i++) {
7             if (mask & (1<<i)) sum += a[i];
8         }
9         if (sum == target) return true;
10    }
11    return false;
12 }
```

Este ejemplo es ingenuo ($\mathcal{O}(n \cdot 2^n)$), pero muestra cómo recorrer subconjuntos con máscaras.

14.5. Técnicas avanzadas

- **Iterar sobre submáscaras:** Para un conjunto *mask*, se pueden recorrer todos sus subconjuntos en $\mathcal{O}(3^n)$ total:

```
1 for (int sub = mask; sub; sub = (sub-1) & mask) {
2     // sub es un submask de mask
3 }
```

- **SOS DP (Sum Over Subsets):** Técnica para precomputar funciones sobre todos los subconjuntos en $\mathcal{O}(n \cdot 2^n)$.

```
1 vector<int> f(1<<n); // valores iniciales
2 for (int i = 0; i < n; i++) {
3     for (int mask = 0; mask < (1<<n); mask++) {
4         if (mask & (1<<i)) {
5             f[mask] += f[mask ^ (1<<i)];
6         }
7     }
8 }
```

- **Bit DP + combinatoria:** Muchos problemas de contar formas (ejemplo: número de matchings perfectos en un grafo bipartito pequeño) se resuelven con DP sobre máscaras.

14.6. Aplicaciones comunes

- TSP y variaciones (Hamiltonian paths).
- Asignación de tareas (Assignment Problem).
- Conteo de matchings en grafos pequeños.
- Subset sum y particiones.
- DP sobre estados de columnas en problemas de rejillas (tiling).

14.7. Ejemplo de uso: Assignment Problem

Asignar n trabajadores a n tareas con costos $cost[i][j]$.

$$dp[mask] = \text{costo mínimo para asignar las tareas en } mask$$

```

1 long long assignment(vector<vector<long long>>& cost) {
2     int n = (int)cost.size();
3     int N = 1 << n;
4     vector<long long> dp(N, 1e15);
5     dp[0] = 0;
6     for (int mask = 0; mask < N; mask++) {
7         int i = __builtin_popcount(mask); // trabajador actual
8         for (int j = 0; j < n; j++) {
9             if (!(mask & (1<<j))) {
10                dp[mask | (1<<j)] = min(dp[mask | (1<<j)],
11                                         dp[mask] + cost[i][j]);
12            }
13        }
14    }
15    return dp[N-1];
16 }

```

15. DP en Árboles

15.1. Descripción general

La **programación dinámica en árboles** (Tree DP) es una técnica que aprovecha la estructura jerárquica de un árbol para resolver problemas de optimización o conteo. La idea central es realizar un recorrido (DFS) y calcular valores para cada nodo en función de sus hijos. Es muy útil porque los árboles no tienen ciclos, lo que permite descomponer el problema de manera natural.

15.2. Patrones comunes de Tree DP

- **DP de subárbol:** calcular una propiedad para el subárbol de cada nodo.
- **DP de rerooting (re-enraizamiento):** calcular valores para todos los nodos como raíz, usando resultados parciales.
- **DP de caminos:** calcular longitudes, sumas o propiedades de caminos en el árbol.
- **DP de conteo:** contar formas de colorear, etiquetar o seleccionar nodos/aristas.

15.3. Ejemplo 1: Tamaño de subárboles

$$dp[u] = \text{número de nodos en el subárbol con raíz en } u$$

```

1 vector<vector<int>> adj;
2 vector<int> subSize;
3
4 void dfs(int u, int p) {

```

```

5     subSize[u] = 1;
6     for (int v : adj[u]) {
7         if (v == p) continue;
8         dfs(v, u);
9         subSize[u] += subSize[v];
10    }
11 }

```

15.4. Ejemplo 2: DP de caminos más largos (diámetro del árbol)

$dp[u]$ = longitud del camino más largo desde u hacia abajo

```

1 int diameter = 0;
2
3 int dfs(int u, int p) {
4     int best1 = 0, best2 = 0;
5     for (int v : adj[u]) {
6         if (v == p) continue;
7         int d = dfs(v, u) + 1;
8         if (d > best1) { best1 = best2; best2 = d; }
9         else if (d > best2) best2 = d;
10    }
11    diameter = max(diameter, best1 + best2);
12    return best1;
13 }

```

15.5. Ejemplo 3: Independent Set en árboles

Problema: seleccionar un conjunto de nodos tal que no haya dos adyacentes, maximizando el número de nodos.

$dp[u][0]$ = máximo tamaño del conjunto independiente en el subárbol de u si u no está incluido

$dp[u][1]$ = máximo tamaño si u está incluido

```

1 vector<vector<int>> adj;
2 vector<vector<int>> dp;
3
4 void dfs(int u, int p) {
5     dp[u][0] = 0;
6     dp[u][1] = 1;
7     for (int v : adj[u]) {
8         if (v == p) continue;
9         dfs(v, u);
10        dp[u][0] += max(dp[v][0], dp[v][1]);

```

```

11         dp[u][1] += dp[v][0];
12     }
13 }

```

15.6. Ejemplo 4: Rerooting DP

Problema: calcular el tamaño del subárbol de cada nodo si fuera la raíz. Se hace en dos fases:

1. DFS para calcular valores de subárboles.
2. DFS de rerooting para propagar información al resto del árbol.

```

1 vector<int> subSize, ans;
2
3 void dfs1(int u, int p) {
4     subSize[u] = 1;
5     for (int v : adj[u]) {
6         if (v == p) continue;
7         dfs1(v, u);
8         subSize[u] += subSize[v];
9     }
10 }
11
12 void dfs2(int u, int p) {
13     ans[u] = subSize[u];
14     for (int v : adj[u]) {
15         if (v == p) continue;
16         int savedU = subSize[u], savedV = subSize[v];
17         subSize[u] -= subSize[v];
18         subSize[v] += subSize[u];
19         dfs2(v, u);
20         subSize[u] = savedU; subSize[v] = savedV;
21     }
22 }

```

15.7. Aplicaciones comunes

- Cálculo de distancias y diámetros.
- Problemas de selección de nodos (independent set, vertex cover).
- Conteo de formas de colorear o etiquetar árboles.
- DP de rerooting para problemas de “respuesta en todos los nodos”.
- Optimización de rutas en árboles (ejemplo: centroid decomposition).

15.8. Ejemplo de uso

```
1 int main() {
2     int n = 5;
3     adj.assign(n, {});
4     adj[0] = {1,2};
5     adj[1] = {0,3,4};
6     adj[2] = {0};
7     adj[3] = {1};
8     adj[4] = {1};
9
10    subSize.assign(n, 0);
11    dfs(0, -1);
12    for (int i = 0; i < n; i++)
13        cout << "Subtree size of " << i << " = " << subSize[i] <<
14        endl;
15 }
```

16. DP en Cadenas: Subsecuencias, Substrings y Palíndromos

16.1. Notas generales de compilación

Todos los ejemplos usan C++11 y requieren las siguientes cabeceras:

```
1 // Cabeceras comunes
2 #include <bits/stdc++.h>
3 using namespace std;
```

Si prefieres algo más explícito:

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <algorithm>
5 using namespace std;
```

16.2. Longest Common Subsequence (LCS)

```
1 int LCS(const string& A, const string& B) {
2     int n = (int)A.size(), m = (int)B.size();
3     vector<vector<int>> dp(n+1, vector<int>(m+1, 0));
4     for (int i = 1; i <= n; i++) {
5         for (int j = 1; j <= m; j++) {
6             if (A[i-1] == B[j-1]) {
7                 dp[i][j] = dp[i-1][j-1] + 1;
8             } else {
9                 dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
10            }
11        }
12    }
13    return dp[n][m];
14 }
```

```

10         }
11     }
12 }
13 return dp[n][m];
14 }

```

16.3. Longest Common Substring

Corregido para asignar 0 explícitamente cuando no coinciden (evita residuos).

```

1 int LCSubstring(const string& A, const string& B) {
2     int n = (int)A.size(), m = (int)B.size();
3     vector<vector<int>> dp(n+1, vector<int>(m+1, 0));
4     int ans = 0;
5     for (int i = 1; i <= n; i++) {
6         for (int j = 1; j <= m; j++) {
7             if (A[i-1] == B[j-1]) {
8                 dp[i][j] = dp[i-1][j-1] + 1;
9                 ans = max(ans, dp[i][j]);
10            } else {
11                dp[i][j] = 0; // correccion: reset explicito
12            }
13        }
14    }
15    return ans;
16 }

```

16.4. Longest Palindromic Subsequence (LPS)

```

1 int LPS(const string& S) {
2     int n = (int)S.size();
3     if (n == 0) return 0;
4     vector<vector<int>> dp(n, vector<int>(n, 0));
5     for (int i = n-1; i >= 0; i--) {
6         dp[i][i] = 1;
7         for (int j = i+1; j < n; j++) {
8             if (S[i] == S[j]) {
9                 dp[i][j] = (j == i+1) ? 2 : dp[i+1][j-1] + 2;
10            } else {
11                dp[i][j] = max(dp[i+1][j], dp[i][j-1]);
12            }
13        }
14    }
15    return dp[0][n-1];
16 }

```


16.5. Longest Palindromic Substring

```
1 string longestPalindrome(const string& S) {
2     int n = (int)S.size();
3     if (n == 0) return "";
4     vector<vector<bool>> dp(n, vector<bool>(n, false));
5     int start = 0, maxLen = 1;
6
7     // substrings de longitud 1
8     for (int i = 0; i < n; i++) dp[i][i] = true;
9
10    // longitud 2..n
11    for (int len = 2; len <= n; len++) {
12        for (int i = 0; i + len - 1 < n; i++) {
13            int j = i + len - 1;
14            if (S[i] == S[j]) {
15                if (len == 2) {
16                    dp[i][j] = true;
17                } else {
18                    dp[i][j] = dp[i+1][j-1];
19                }
20                if (dp[i][j] && len > maxLen) {
21                    start = i;
22                    maxLen = len;
23                }
24            } else {
25                dp[i][j] = false; // explicito por claridad
26            }
27        }
28    }
29    return S.substr(start, maxLen);
30 }
```

16.6. Edit Distance (Levenshtein)

Incluye corrección de cabeceras y tipos, con inicialización robusta.

```
1 int editDistance(const string& A, const string& B) {
2     int n = (int)A.size(), m = (int)B.size();
3     vector<vector<int>> dp(n+1, vector<int>(m+1, 0));
4
5     for (int i = 0; i <= n; i++) dp[i][0] = i;
6     for (int j = 0; j <= m; j++) dp[0][j] = j;
7
8     for (int i = 1; i <= n; i++) {
9         for (int j = 1; j <= m; j++) {
10             if (A[i-1] == B[j-1]) {
11                 dp[i][j] = dp[i-1][j-1];
12             } else {
```

```

13         dp[i][j] = 1 + min({dp[i-1][j],          // eliminar
14                             dp[i][j-1],          // insertar
15                             dp[i-1][j-1]        // reemplazar
16                             });
17     }
18 }
19 }
20 return dp[n][m];
21 }

```

16.7. Notas de optimización y robustez

- **Espacio:** LCS y edit distance pueden reducirse a 2 filas ($\mathcal{O}(m)$) si solo necesitas la longitud.
- **Casos vacíos:** usa retornos tempranos para $n = 0$ o $m = 0$; ya están contemplados.
- **Indices:** siempre trabaja en prefijos 1-indexados para DP en dos cadenas; simplifica bordes.
- **Reconstrucción:** guarda decisiones (flechas) para recuperar la solución (LCS/LPS).

17. Geometría Computacional General

17.1. Descripción general

La geometría computacional estudia algoritmos y estructuras para resolver problemas geométricos en el plano o el espacio. Aunque muchos problemas requieren técnicas avanzadas, existe un conjunto de herramientas generales, fórmulas y abstracciones que son útiles en una gran variedad de contextos competitivos.

17.2. Representación de puntos y vectores

```

1 struct Point {
2     double x, y;
3     Point operator+(const Point& p) const { return {x + p.x, y + p
4     .y}; }
5     Point operator-(const Point& p) const { return {x - p.x, y - p
6     .y}; }
7     Point operator*(double k) const { return {x * k, y * k}; }
8     Point operator/(double k) const { return {x / k, y / k}; }
9 };

```

17.3. Operaciones básicas

- **Producto escalar (dot product):**

$$a \cdot b = a_x b_x + a_y b_y$$

Útil para proyecciones y ángulos.

- **Producto cruzado (cross product):**

$$a \times b = a_x b_y - a_y b_x$$

Útil para orientación, área y perpendicularidad.

- **Norma (magnitud):**

$$\|a\| = \sqrt{a_x^2 + a_y^2}$$

- **Distancia entre puntos:**

$$d(p, q) = \sqrt{(p_x - q_x)^2 + (p_y - q_y)^2}$$

- **Ángulo entre vectores:**

$$\theta = \arccos \left(\frac{a \cdot b}{\|a\| \cdot \|b\|} \right)$$

17.4. Orientación de tres puntos

Determina si los puntos a , b , c están en sentido horario, antihorario o colineales:

$$\text{orient}(a, b, c) = (b - a) \times (c - a)$$

- > 0 : antihorario
- < 0 : horario
- $= 0$: colineales

17.5. Área de un polígono (fórmula de Shoelace)

Dado un polígono simple con vértices $(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})$:

$$A = \frac{1}{2} \left| \sum_{i=0}^{n-1} x_i y_{i+1} - x_{i+1} y_i \right|$$

con índices módulo n .

17.6. Distancia punto a recta

Dado punto p y recta definida por puntos a y b :

$$d = \frac{|(b-a) \times (p-a)|}{\|b-a\|}$$

17.7. Proyección de punto sobre recta

$$\text{proj}_{ab}(p) = a + \frac{(p-a) \cdot (b-a)}{\|b-a\|^2} \cdot (b-a)$$

17.8. Circunferencia

- Ecuación estándar:

$$(x-x_0)^2 + (y-y_0)^2 = r^2$$

- Distancia entre centro y punto:

$$d = \sqrt{(x-x_0)^2 + (y-y_0)^2}$$

- Intersección de dos círculos: Resolver sistema de ecuaciones cuadráticas.

17.9. Triángulos

- Área por Herón:

$$s = \frac{a+b+c}{2}, \quad A = \sqrt{s(s-a)(s-b)(s-c)}$$

- **Circuncentro:** Punto equidistante de los vértices, intersección de mediatrices.
- **Baricentro:** Promedio de los vértices:

$$G = \frac{A+B+C}{3}$$

- **Ortocentro:** Intersección de alturas.

17.10. Intersección de segmentos

Dos segmentos $[a, b]$ y $[c, d]$ se intersectan si:

$$\text{orient}(a, b, c) \cdot \text{orient}(a, b, d) \leq 0 \quad \text{y} \quad \text{orient}(c, d, a) \cdot \text{orient}(c, d, b) \leq 0$$

17.11. Convexidad

Un polígono es convexo si todos los giros entre triples consecutivos de vértices tienen la misma orientación.

17.12. Métodos útiles

- **Convex Hull (envolvente convexa):** Graham Scan, Andrew's monotone chain — ordena puntos y construye envolvente en $\mathcal{O}(n \log n)$.
- **Rotating Calipers:** Encuentra el diámetro de un conjunto de puntos, pares más lejanos, etc.
- **Sweep Line:** Detecta intersecciones, calcula áreas, ordena eventos en el plano.
- **Segment Tree + Geometry:** Para mantener segmentos activos en tiempo real (ejemplo: intersección de segmentos).
- **Binary Search sobre ángulos o distancias:** Útil en problemas de optimización geométrica.

17.13. Precisión numérica

- Usar `double` con cuidado: errores acumulativos.
- Comparar con tolerancia:

$$\text{igual}(a, b) = |a - b| < \varepsilon$$

- Para precisión exacta: usar enteros o fracciones racionales si el problema lo permite.

18. Conteo Combinatorio con Programación Dinámica

18.1. Descripción general

Muchos problemas de conteo pueden resolverse con **DP combinatoria**, donde el estado representa el número de formas de construir una estructura bajo ciertas restricciones. La clave está en definir correctamente los estados, transiciones y condiciones base, y en aprovechar propiedades combinatorias como factoriales, binomios, particiones y principios de inclusión-exclusión.

18.2. Técnicas fundamentales

- **DP directa:** contar formas de construir objetos paso a paso.
- **DP con combinatoria precomputada:** usar combinaciones, factoriales, permutaciones.
- **DP con inclusión-exclusión:** contar con restricciones (evitar repeticiones, condiciones).
- **DP con memoización:** evitar recomputación en recursiones combinatorias.

18.3. Precomputación combinatoria

18.3.1. Factoriales y combinaciones

$$\text{Combinación: } \binom{n}{k} = \frac{n!}{k!(n-k)!}$$

```
1 const int MOD = 1e9+7, N = 1e5;
2 vector<long long> fact(N+1), invFact(N+1);
3
4 long long modPow(long long a, long long b) {
5     long long res = 1;
6     while (b) {
7         if (b & 1) res = res * a % MOD;
8         a = a * a % MOD;
9         b >>= 1;
10    }
11    return res;
12 }
13
14 void precompute() {
15     fact[0] = 1;
16     for (int i = 1; i <= N; i++) fact[i] = fact[i-1] * i % MOD;
17     invFact[N] = modPow(fact[N], MOD-2);
18     for (int i = N-1; i >= 0; i--) invFact[i] = invFact[i+1] * (i
19         +1) % MOD;
20 }
21 long long C(int n, int k) {
22     if (k < 0 || k > n) return 0;
23     return fact[n] * invFact[k] % MOD * invFact[n-k] % MOD;
24 }
```

18.4. Ejemplo 1: Número de formas de sumar a n con enteros positivos

$$dp[n] = \text{número de formas de escribir } n = a_1 + a_2 + \dots + a_k$$

```
1 int countPartitions(int n) {
2     vector<int> dp(n+1, 0);
3     dp[0] = 1;
4     for (int i = 1; i <= n; i++) {
5         for (int j = i; j <= n; j++) {
6             dp[j] += dp[j - i];
7         }
8     }
9     return dp[n];
10 }
```

18.5. Ejemplo 2: Número de arreglos válidos (permutaciones con restricciones)

Problema: contar permutaciones de n elementos donde ningún elemento esté en su posición original (derangements).

$$D_n = (n-1)(D_{n-1} + D_{n-2})$$

$$D_0 = 1, \quad D_1 = 0$$

```
1 int derangements(int n) {
2     vector<long long> D(n+1);
3     D[0] = 1; D[1] = 0;
4     for (int i = 2; i <= n; i++) {
5         D[i] = (i-1) * (D[i-1] + D[i-2]) % MOD;
6     }
7     return D[n];
8 }
```

18.6. Ejemplo 3: Número de formas de colocar k objetos en n cajas sin vacías

$S(n, k)$ = número de formas de dividir k objetos en n grupos no vacíos

(Stirling de segunda especie)

$$S(n, k) = S(n-1, k-1) + k \cdot S(n-1, k)$$

```
1 int stirling2(int n, int k) {
2     vector<vector<long long>> S(n+1, vector<long long>(k+1, 0));
3     S[0][0] = 1;
4     for (int i = 1; i <= n; i++) {
5         for (int j = 1; j <= k; j++) {
6             S[i][j] = (S[i-1][j-1] + j * S[i-1][j]) % MOD;
7         }
8     }
9     return S[n][k];
10 }
```

18.7. Ejemplo 4: Caminos en una cuadrícula

Número de caminos de $(0, 0)$ a (n, m) moviendo solo derecha y abajo:

$$\text{Total} = \binom{n+m}{n}$$

Si hay obstáculos, usar DP:

```

1 int gridPaths(int n, int m, vector<vector<bool>>& blocked) {
2     vector<vector<int>> dp(n, vector<int>(m, 0));
3     dp[0][0] = blocked[0][0] ? 0 : 1;
4     for (int i = 0; i < n; i++) {
5         for (int j = 0; j < m; j++) {
6             if (blocked[i][j]) continue;
7             if (i > 0) dp[i][j] += dp[i-1][j];
8             if (j > 0) dp[i][j] += dp[i][j-1];
9             dp[i][j] %= MOD;
10        }
11    }
12    return dp[n-1][m-1];
13 }

```

18.8. Principios combinatorios útiles

- **Principio de multiplicación:** si hay a formas de hacer una cosa y b formas de otra, hay $a \cdot b$ formas de hacer ambas.
- **Principio de inclusión-exclusión:**

$$|A \cup B| = |A| + |B| - |A \cap B|$$

Generalizable a más conjuntos.

- **Identidades combinatorias:**

$$\sum_{k=0}^n \binom{n}{k} = 2^n, \quad \sum_{k=0}^n k \cdot \binom{n}{k} = n \cdot 2^{n-1}$$

- **Catalan numbers:**

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

Aparecen en estructuras balanceadas: árboles binarios, paréntesis válidos, etc.

19. Backtracking y Perfección de la Fuerza Bruta

19.1. Descripción general

El **backtracking** es una técnica de búsqueda sistemática que explora todas las configuraciones posibles de una solución, descartando aquellas que violan las restricciones del problema. Es una forma estructurada de fuerza bruta, donde se evita explorar ramas inútiles del espacio de búsqueda mediante **poda**.

19.2. Esquema general

```
1 void backtrack(Estado actual) {
2     if (esSolucion(actual)) {
3         procesar(actual);
4         return;
5     }
6     for (opcion : opcionesDisponibles(actual)) {
7         if (esValida(opcion, actual)) {
8             aplicar(opcion, actual);
9             backtrack(actual);
10            deshacer(opcion, actual);
11        }
12    }
13 }
```

19.3. Técnicas para perfeccionar fuerza bruta

- **Poda (pruning):** evitar ramas que no pueden llevar a una solución mejor.
- **Ordenamiento heurístico:** probar primero las opciones más prometedoras.
- **Memoización parcial:** recordar subestados ya explorados.
- **Bitmasks:** representar subconjuntos de forma eficiente.
- **Simetrías:** evitar explorar configuraciones equivalentes.
- **Branch and Bound:** mantener un límite superior/inferior para cortar ramas.

19.4. Ejemplo 1: N-Reinas

```
1 int n, soluciones = 0;
2 vector<bool> col, diag1, diag2;
3
4 void dfs(int fila) {
5     if (fila == n) {
6         soluciones++;
7         return;
8     }
9     for (int c = 0; c < n; c++) {
10        if (col[c] || diag1[fila+c] || diag2[fila-c+n-1]) continue
11        ;
12        col[c] = diag1[fila+c] = diag2[fila-c+n-1] = true;
13        dfs(fila+1);
14        col[c] = diag1[fila+c] = diag2[fila-c+n-1] = false;
15    }
16 }
```

```

17 int nQueens(int _n) {
18     n = _n;
19     col.assign(n, false);
20     diag1.assign(2*n, false);
21     diag2.assign(2*n, false);
22     soluciones = 0;
23     dfs(0);
24     return soluciones;
25 }

```

19.5. Ejemplo 2: Generar todas las permutaciones

```

1 void permute(vector<int>& a, int l) {
2     if (l == a.size()) {
3         procesar(a);
4         return;
5     }
6     for (int i = l; i < a.size(); i++) {
7         swap(a[i], a[l]);
8         permute(a, l+1);
9         swap(a[i], a[l]);
10    }
11 }

```

19.6. Ejemplo 3: Subconjuntos con suma objetivo

```

1 void subsetSum(vector<int>& a, int i, int suma, int objetivo) {
2     if (i == a.size()) {
3         if (suma == objetivo) contar++;
4         return;
5     }
6     subsetSum(a, i+1, suma, objetivo);
7     subsetSum(a, i+1, suma + a[i], objetivo);
8 }

```

19.7. Aplicaciones comunes

- Generación de combinaciones, permutaciones, subconjuntos.
- Problemas de asignación (satisfacción de restricciones).
- Juegos de tablero (sudoku, ajedrez, rompecabezas).
- Búsqueda de soluciones óptimas en espacios pequeños.
- Problemas de partición, coloreo, emparejamiento.

19.8. Consejos prácticos

- Usa `return` temprano si detectas que no se puede continuar.
- Usa estructuras auxiliares para evitar recomputación (memoización).
- Usa `bitmasks` para representar estados de forma eficiente.
- Si el orden no importa, evita permutaciones redundantes (ej. usando sets o marcando elementos).

20. Aritmética Modular y Teoría de Números

20.1. Aritmética modular

Aritmética modular estudia operaciones bajo un módulo m , es decir, se trabaja con residuos:

$$a \equiv b \pmod{m} \iff m \mid (a - b)$$

20.1.1. Propiedades básicas

$$\begin{aligned}(a + b) \pmod{m} &= (a \pmod{m} + b \pmod{m}) \pmod{m} \\(a - b) \pmod{m} &= (a \pmod{m} - b \pmod{m} + m) \pmod{m} \\(a \cdot b) \pmod{m} &= (a \pmod{m} \cdot b \pmod{m}) \pmod{m} \\(a^k) \pmod{m} &= \text{usar exponenciación rápida}\end{aligned}$$

20.1.2. Inverso modular

El inverso de a módulo m es el número a^{-1} tal que:

$$a \cdot a^{-1} \equiv 1 \pmod{m}$$

Existe si y solo si $\gcd(a, m) = 1$.

Cálculo:

- Si m es primo:

$$a^{-1} \equiv a^{m-2} \pmod{m}$$

(por el pequeño teorema de Fermat)

- En general: usar el **algoritmo extendido de Euclides**.

```

1 long long modInverse(long long a, long long m) {
2     long long b = m, x = 1, y = 0;
3     while (b) {
4         long long q = a / b;
5         tie(a, b) = make_pair(b, a % b);
6         tie(x, y) = make_pair(y, x - q * y);
7     }
8     return (x % m + m) % m;
9 }

```

20.2. Exponenciación rápida

a^b mód m en $\mathcal{O}(\log b)$

```

1 long long modPow(long long a, long long b, long long m) {
2     long long res = 1;
3     a %= m;
4     while (b) {
5         if (b & 1) res = res * a % m;
6         a = a * a % m;
7         b >>= 1;
8     }
9     return res;
10 }

```

20.3. Pequeño teorema de Fermat

Si p es primo y $a \not\equiv 0 \pmod{p}$:

$$a^{p-1} \equiv 1 \pmod{p} \Rightarrow a^{-1} \equiv a^{p-2} \pmod{p}$$

20.4. Teorema de Euler

Si $\phi(m)$ es la función totiente de Euler:

$$a^{\phi(m)} \equiv 1 \pmod{m} \quad \text{si } \gcd(a, m) = 1$$

20.4.1. Función totiente

$$\phi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

20.5. Criba de Eratóstenes

```
1 vector<bool> isPrime(int n) {
2     vector<bool> prime(n+1, true);
3     prime[0] = prime[1] = false;
4     for (int i = 2; i*i <= n; i++) {
5         if (prime[i]) {
6             for (int j = i*i; j <= n; j += i)
7                 prime[j] = false;
8         }
9     }
10    return prime;
11 }
```

20.6. Factorización en primos

```
1 vector<int> primeFactors(int n) {
2     vector<int> factors;
3     for (int i = 2; i*i <= n; i++) {
4         while (n % i == 0) {
5             factors.push_back(i);
6             n /= i;
7         }
8     }
9     if (n > 1) factors.push_back(n);
10    return factors;
11 }
```

20.7. Conteo de divisores

Si $n = p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k}$, entonces: $\tau(n) = (e_1 + 1)(e_2 + 1) \cdots (e_k + 1)$

20.8. Suma de divisores

$$\sigma(n) = \prod_{i=1}^k \left(\frac{p_i^{e_i+1} - 1}{p_i - 1} \right)$$

20.9. Aplicaciones comunes

- Cálculo de combinaciones módulo primo.
- Inversos para división modular.
- Criptografía (RSA, Diffie-Hellman).

- Problemas de conteo con restricciones modulares.
- Teoremas clásicos: Fermat, Euler, Wilson, Chino del Resto.

21. Operaciones Bit a Bit y Aplicaciones

21.1. Descripción general

Las operaciones bit a bit permiten manipular directamente los bits de enteros. Son extremadamente rápidas y útiles en programación competitiva, especialmente en problemas de máscaras, optimización, conteo y simulación de estados.

21.2. Operaciones básicas

- AND: $a \& b$ — bit 1 si ambos bits son 1.
- OR: $a \mid b$ — bit 1 si al menos uno es 1.
- XOR: $a \wedge b$ — bit 1 si los bits son distintos.
- NOT: $\sim a$ — invierte todos los bits.
- SHIFT LEFT: $a \ll k$ — multiplica por 2^k .
- SHIFT RIGHT: $a \gg k$ — divide por 2^k (sin signo).

21.3. Operaciones útiles

- $a \& (a - 1)$: elimina el bit más bajo encendido.
- $a \& -a$: aísla el bit más bajo encendido.
- $a \mid (a - 1)$: enciende todos los bits hasta el más bajo encendido.
- $a \wedge (a - 1)$: máscara con todos los bits hasta el más bajo encendido.
- $a \& (1 \ll k)$: verifica si el bit k está encendido.
- $a \mid (1 \ll k)$: enciende el bit k .
- $a \& \sim(1 \ll k)$: apaga el bit k .
- `__builtin_popcount(a)`: número de bits encendidos (solo en GCC/Clang).
- `__builtin_ctz(a)`: ceros al final (posición del bit más bajo encendido).
- `__builtin_clz(a)`: ceros al inicio (posición del bit más alto encendido).

21.4. Aplicaciones comunes

- **Bitmask DP:** representar subconjuntos como enteros.
- **Enumeración de subconjuntos:** recorrer todos los submasks.
- **Conteo de bits:** problemas de paridad, distancia Hamming.
- **Simulación de estados:** juegos, tableros, configuraciones.
- **Compresión de información:** representar múltiples flags en un entero.
- **Optimización de espacio:** usar enteros en lugar de arreglos booleanos.

21.5. Ejemplo: recorrer todos los subconjuntos de una máscara

```
1 int mask = 0b101101;  
2 for (int sub = mask; sub; sub = (sub - 1) & mask) {  
3     procesar(sub);  
4 }
```

21.6. Ejemplo: contar bits encendidos

```
1 int countBits(int x) {  
2     int cnt = 0;  
3     while (x) {  
4         x &= (x - 1);  
5         cnt++;  
6     }  
7     return cnt;  
8 }
```

21.7. Ejemplo: encontrar el bit más bajo encendido

```
1 int lowestBit(int x) {  
2     return x & -x;  
3 }
```

21.8. Ejemplo: generar todas las combinaciones de k bits encendidos en n posiciones

```
1 void generateCombinations(int n, int k) {  
2     int comb = (1 << k) - 1;  
3     while (comb < (1 << n)) {  
4         procesar(comb);  
5         int x = comb & -comb, y = comb + x;
```

```

6         comb = ((comb & ~y) / x >> 1) | y;
7     }
8 }

```

21.9. Consejos prácticos

- Usa `bitset<N>` si necesitas manipular muchos bits (hasta miles).
- Usa máscaras para representar estados en DP, juegos, configuraciones.
- Usa operaciones bit a bit para acelerar simulaciones y evitar estructuras pesadas.
- Usa `popcount` para contar elementos en subconjuntos.
- Usa `ctz/clz` para encontrar posiciones de bits activos.

22. Fórmulas Generales y Teoremas Clásicos

22.1. Áreas y perímetros

- Cuadrado:

$$A = a^2, \quad P = 4a$$

- Rectángulo:

$$A = ab, \quad P = 2(a + b)$$

- Triángulo:

$$A = \frac{1}{2}bh, \quad A = \frac{1}{2}ab \sin C$$

- Triángulo (Herón):

$$s = \frac{a + b + c}{2}, \quad A = \sqrt{s(s - a)(s - b)(s - c)}$$

- Círculo:

$$A = \pi r^2, \quad C = 2\pi r$$

- Trapecio:

$$A = \frac{1}{2}(a + b)h$$

- Polígono regular de n lados:

$$A = \frac{n}{4}a^2 \cot\left(\frac{\pi}{n}\right)$$

- Área por coordenadas (Shoelace):

$$A = \frac{1}{2} \left| \sum_{i=0}^{n-1} x_i y_{i+1} - x_{i+1} y_i \right|$$

22.2. Trigonometría esencial

- Ley de senos:

$$\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C} = 2R$$

- Ley de cosenos:

$$c^2 = a^2 + b^2 - 2ab \cos C$$

- Identidades:

$$\sin^2 x + \cos^2 x = 1, \quad \tan x = \frac{\sin x}{\cos x}$$

22.3. Álgebra y combinatoria

- Binomio de Newton:

$$(a + b)^n = \sum_{k=0}^n \binom{n}{k} a^{n-k} b^k$$

- Suma de los primeros n enteros:

$$\sum_{k=1}^n k = \frac{n(n+1)}{2}$$

- Suma de cuadrados:

$$\sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6}$$

- Suma de cubos:

$$\sum_{k=1}^n k^3 = \left(\frac{n(n+1)}{2} \right)^2$$

- Identidades notables:

$$(a \pm b)^2 = a^2 \pm 2ab + b^2, \quad a^2 - b^2 = (a - b)(a + b)$$

22.4. Teoremas clásicos

- Teorema de Pitágoras:

$$c^2 = a^2 + b^2$$

- Teorema de Thales: Si A , B , C están en un círculo y AC es diámetro, entonces $\angle ABC = 90^\circ$.

- Teorema de Menelao (triángulo con transversal):

$$\frac{AF}{FB} \cdot \frac{BD}{DC} \cdot \frac{CE}{EA} = 1$$

- Teorema de Ceva (conurrencia de cevianas):

$$\frac{BD}{DC} \cdot \frac{CE}{EA} \cdot \frac{AF}{FB} = 1$$

- Teorema de Pick (área de polígono en grilla):

$$A = I + \frac{B}{2} - 1$$

donde I = puntos interiores, B = puntos en el borde.

22.5. Fórmulas de conteo

- Permutaciones:

$$P(n, k) = \frac{n!}{(n - k)!}$$

- Combinaciones:

$$C(n, k) = \binom{n}{k} = \frac{n!}{k!(n - k)!}$$

- Combinaciones con repetición:

$$\binom{n + k - 1}{k}$$

- Particiones de un entero n :

$p(n)$ = número de formas de escribir n como suma de enteros positivos

- Números de Catalan:

$$C_n = \frac{1}{n + 1} \binom{2n}{n}$$

22.6. Fórmulas físicas discretas útiles

- Movimiento rectilíneo uniforme:

$$d = vt$$

- Movimiento uniformemente acelerado:

$$d = v_0 t + \frac{1}{2} a t^2, \quad v = v_0 + a t$$

- Energía cinética:

$$E_k = \frac{1}{2} m v^2$$

- Ley de gravitación universal:

$$F = G \frac{m_1 m_2}{r^2}$$

22.7. Constantes útiles

- $\pi \approx 3,14159265$
- $e \approx 2,7182818$
- $\phi = \frac{1+\sqrt{5}}{2} \approx 1,618$
- $G = 6,674 \times 10^{-11} \text{ Nm}^2/\text{kg}^2$

22.8. Aplicaciones comunes

- Conteo de inversiones.
- Mantenimiento de frecuencias y orden estadístico (k-ésimo menor).
- Actualizaciones puntuales con consultas de rango, o transformaciones para actualizaciones de rango con consultas de punto/rango.
- Barridos en problemas geométricos (sweep line) con compresión de coordenadas.
- Mantenimiento de prefijos en arreglos y en 2D.

22.9. Detalles, variantes y consideraciones avanzadas

- **Indexación 1-based:** El BIT funciona de manera natural con índices que inician en 1. Si tu arreglo es 0-based, desplaza los índices.
- **Overflow:** Para sumar frecuencias o valores grandes, usa `long long`.
- **Operaciones invertibles:** El BIT depende de una operación de grupo abeliano (asociativa e invertible) para que las actualizaciones y prefijos sean coherentes. Suma y XOR funcionan; `min`/`max` requieren cuidado y no son simétricas para obtener rangos arbitrarios.
- **Bulk build en $O(n)$:** Si tienes el arreglo inicial, el método `build` mostrado construye el árbol en $O(n)$. Alternativamente, llama `add(i, arr[i])` para cada `i` en $O(n \log n)$.
- **Two-BIT trick para `range update` + `range query`:** Usa dos BITs con la transformación lineal presentada; es estándar en problemas que requieren agregar constantes a rangos y consultar sumas de rangos.
- **Fenwick interactivo:** Para recuperar el valor de un punto (cuando se mantienen prefijos) usa `sumRange(i, i)`; si hay *range updates* con *point queries*, consulta el prefijo del índice.
- **Generalización:** Se puede parametrizar la operación (suma, xor) con plantillas y functors si se necesita reutilización en múltiples problemas.

23. Tablas de Complejidad

23.1. Estructuras estándar

- `vector`:
 - Acceso por índice: $\mathcal{O}(1)$
 - Inserción al final: $\mathcal{O}(1)$ amortizado
 - Inserción en medio: $\mathcal{O}(n)$
 - Búsqueda lineal: $\mathcal{O}(n)$
- `deque`:
 - Acceso por índice: $\mathcal{O}(1)$
 - Inserción al inicio/fin: $\mathcal{O}(1)$ amortizado
 - Inserción en medio: $\mathcal{O}(n)$
- `set` / `map` (basado en árboles balanceados):
 - Inserción: $\mathcal{O}(\log n)$
 - Búsqueda: $\mathcal{O}(\log n)$
 - Eliminación: $\mathcal{O}(\log n)$
- `unordered_set` / `unordered_map` (hashing):
 - Inserción: $\mathcal{O}(1)$ promedio
 - Búsqueda: $\mathcal{O}(1)$ promedio
 - Eliminación: $\mathcal{O}(1)$ promedio
- `priority_queue`:
 - Inserción: $\mathcal{O}(\log n)$
 - Acceso al máximo: $\mathcal{O}(1)$
 - Eliminación del máximo: $\mathcal{O}(\log n)$
- `stack` / `queue`:
 - Inserción y eliminación: $\mathcal{O}(1)$
 - Acceso al tope/frente: $\mathcal{O}(1)$

23.2. Algoritmos comunes

- Búsqueda lineal: $\mathcal{O}(n)$
- Búsqueda binaria: $\mathcal{O}(\log n)$
- Ordenamiento (`std::sort`): $\mathcal{O}(n \log n)$
- DFS/BFS en grafo: $\mathcal{O}(n + m)$
- Dijkstra con heap: $\mathcal{O}(m \log n)$
- Floyd-Warshall: $\mathcal{O}(n^3)$
- DP simple: $\mathcal{O}(n)$ a $\mathcal{O}(n^2)$
- DP con bitmasks: $\mathcal{O}(n \cdot 2^n)$
- Segment tree: construcción $\mathcal{O}(n)$, consulta/actualización $\mathcal{O}(\log n)$

24. Fórmulas Matemáticas

24.1. Sumatorias clásicas

- Suma de los primeros n enteros:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

- Suma de los primeros n cuadrados:

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

- Suma de los primeros n cubos:

$$\sum_{i=1}^n i^3 = \left(\frac{n(n+1)}{2} \right)^2$$

24.2. Identidades algebraicas

- Binomio cuadrado:

$$(a \pm b)^2 = a^2 \pm 2ab + b^2$$

- Diferencia de cuadrados:

$$a^2 - b^2 = (a - b)(a + b)$$

- Binomio de Newton:

$$(a + b)^n = \sum_{k=0}^n \binom{n}{k} a^{n-k} b^k$$

24.3. Fórmulas geométricas

- Área de triángulo (base y altura):

$$A = \frac{1}{2} \cdot \text{base} \cdot \text{altura}$$

- Área de triángulo (Herón):

$$s = \frac{a + b + c}{2}, \quad A = \sqrt{s(s-a)(s-b)(s-c)}$$

- Área de círculo:

$$A = \pi r^2$$

- Perímetro de círculo:

$$P = 2\pi r$$

- Área de polígono regular de n lados:

$$A = \frac{n}{4} a^2 \cot\left(\frac{\pi}{n}\right)$$

- Área de polígono por coordenadas (Shoelace):

$$A = \frac{1}{2} \left| \sum_{i=0}^{n-1} x_i y_{i+1} - x_{i+1} y_i \right|$$

24.4. Combinatoria

- Permutaciones:

$$P(n, k) = \frac{n!}{(n-k)!}$$

- Combinaciones:

$$C(n, k) = \binom{n}{k} = \frac{n!}{k!(n-k)!}$$

- Combinaciones con repetición:

$$\binom{n+k-1}{k}$$

- Números de Catalan:

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

24.5. Trigonometría

- Ley de senos:

$$\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C}$$

- Ley de cosenos:

$$c^2 = a^2 + b^2 - 2ab \cos C$$

- Identidad fundamental:

$$\sin^2 x + \cos^2 x = 1$$

24.6. Teoría de números

- Pequeño teorema de Fermat:

$$a^{p-1} \equiv 1 \pmod{p} \quad \text{si } p \text{ primo y } a \not\equiv 0$$

- Teorema de Euler:

$$a^{\phi(n)} \equiv 1 \pmod{n} \quad \text{si } \gcd(a, n) = 1$$

- Función totiente de Euler:

$$\phi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

25. Cómo abordar problemas algorítmicos

25.1. 1. Leer con atención

Antes de pensar en código, comprende el problema:

- ¿Qué se pide exactamente?
- ¿Qué tipo de salida se espera?
- ¿Cuáles son las restricciones de entrada?
- ¿Hay casos límite o triviales?

25.2. 2. Identificar la categoría del problema

Clasifica el problema:

- ¿Es de conteo, optimización, simulación, búsqueda, geometría?
- ¿Involucra estructuras de datos, DP, grafos, teoría de números?
- ¿Es un problema clásico disfrazado?

25.3. 3. Analizar restricciones

Esto determina la técnica:

- $n \leq 10$: fuerza bruta o backtracking.
- $n \leq 20$: DP con bitmasks.
- $n \leq 1000$: DP con $\mathcal{O}(n^2)$.
- $n \leq 10^5$: algoritmos $\mathcal{O}(n \log n)$.
- $n \leq 10^9$: matemáticas, búsqueda binaria, greedy.

25.4. 4. Pensar en estados y transiciones

Si es DP:

- ¿Qué define un estado?
- ¿Cómo se transiciona entre estados?
- ¿Cuál es el caso base?

Si es búsqueda:

- ¿Qué representa un nodo?
- ¿Cómo se generan vecinos?
- ¿Cómo se evita visitar estados repetidos?

25.5. 5. Escribir código limpio y modular

- Usa funciones auxiliares para claridad.
- Usa nombres descriptivos.
- Prueba con casos pequeños antes de enviar.

25.6. 6. Depurar con propósito

- Imprime estados intermedios.
- Usa asserts para verificar invariantes.
- Compara con soluciones lentas o fuerza bruta.

25.7. 7. Optimizar solo si es necesario

Primero hazlo funcionar, luego hazlo rápido. Identifica cuellos de botella con perfiles o conteo de llamadas.

25.8. 8. Aprender del problema

Después de resolver:

- ¿Qué técnica fue clave?
- ¿Qué patrón se repite?
- ¿Cómo lo generalizarías?